

# Créer des logiciels via les spécifications

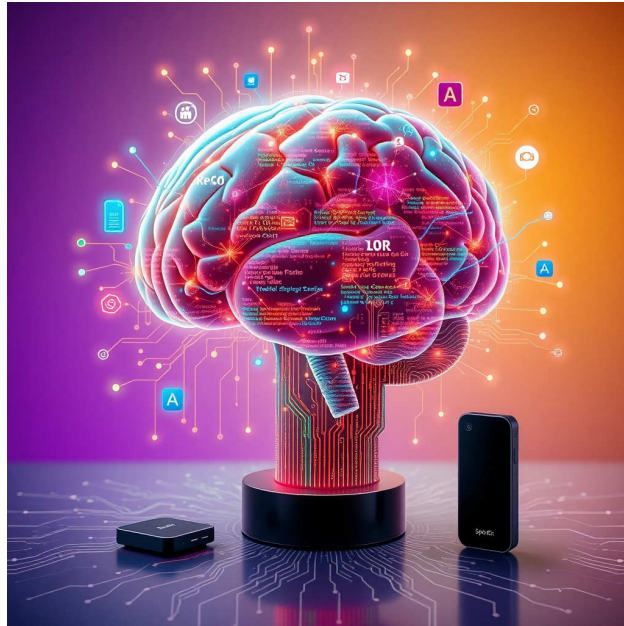


Figure 1: Spec-Kit IA : créer logiciels via spécifications

1. Pourquoi le code seul ne suffit plus
2. Spécifications exécutables qui guident l'IA
3. Cycle Spec Plan Tasks Implement expliqué
4. Prompts ad hoc contre contexte structuré
5. Historique et version 1.0 communautaire
6. Agents compatibles et intégrations disponibles
7. Presets extensions workflows et bundles
8. Cas d'usage logiciels et au-delà
9. Prérequis uv sur Windows macOS Linux
10. Installation via dépôt Git et PyPI
11. Vérifier mise à jour et dépannage
12. Préparer un agent IA de codage
13. Initialiser un projet avec intégration agent
14. Comprendre dossiers et fichiers générés
15. Mode non interactif pour CI et automatisation
16. Changer d'agent sans verrouillage
17. Rôle du fichier constitution du projet
18. Rédiger principes testabilité et qualité
19. Faire modifier la constitution par l'IA
20. Valider les règles avant de spécifier
21. Décrire le besoin en langage naturel
22. Structurer scénarios utilisateurs et exigences
23. Lever les besoins de clarification
24. Valider une spécification prête à planifier
25. Choisir technologies sans deviner tôt
26. Générer plan recherche et modèle de données
27. Définir contrats API et accès fichiers
28. Analyser cohérence spec plan et qualité
29. Transformer le plan en tâches actionnables
30. Ordonner dépendances et jalons testables
31. Estimer complexité et risques techniques
32. Réviser la liste des tâches avec l'IA
33. Exécuter les tâches étape par étape
34. Garder l'IA alignée sur fichiers sources
35. Gérer le contexte et éviter les dérives
36. Suivre l'avancement dans le projet
37. Vérifier implémentation contre spec et plan
38. Répéter implémentation jusqu'à convergence
39. Ajouter garde-fous architecture et CI
40. Documenter la livraison finale

41. Installer l'extension de correction de bugs
42. Évaluer un rapport et isoler la cause
43. Appliquer un correctif ciblé et tracé
44. Tester et valider la résolution
45. Installer l'extension d'évaluation d'idées
46. Collecter et rechercher preuves pour contre
47. Définir problème objectifs et indicateurs
48. Façonner options puis décider go ou stop
49. Créer presets extensions et workflows
50. Explorer processus AIDE Canon Product Forge
51. Publier et partager un bundle réutilisable
52. Automatiser de bout en bout en flux guidé
53. Héberger catalogues privés et recommandations
54. Travailler hors ligne derrière pare-feu
55. Gouverner conformité et bonnes pratiques
56. Adopter Spec-Kit sur projet existant

# 1. Pourquoi le code seul ne suffit plus

## Le problème du code écrit trop tôt

Le **code seul** ne suffit plus quand une IA génère vite, mais sans comprendre finement le besoin réel. Un programme peut compiler, passer des tests partiels et rester pourtant *incorrect* du point de vue métier, car il répond à une intention mal formulée ou incomplète.

Dans ce contexte, le risque principal n'est pas l'absence de code, mais l'**écart entre l'idée et l'implémentation**. Plus ce décalage est grand, plus l'équipe découvre tard les erreurs de logique, les oublis fonctionnels ou les comportements inattendus.

## Pourquoi une spécification change la méthode

Une **spécification** est une description claire de *ce qu'il faut construire* et *pourquoi*, avant de parler de la manière technique de le faire. GitHub Spec Kit place cette description comme **source de vérité** du projet, au lieu de laisser le code devenir la première version "réelle" de l'idée.

Cette approche évite le piège du *vibe coding*, où l'on produit du code rapidement à partir d'un prompt vague. Spec Kit pousse au contraire à commencer par le besoin métier, puis à faire évoluer la solution de façon structurée.

Une bonne spécification agit comme un contrat lisible entre humains et IA

## Ce que le code ne peut pas deviner

Le code ne sait pas seul distinguer une demande *fonctionnellement correcte* d'une demande seulement *techniquement plausible*. Par exemple, "créer un formulaire de réservation" peut signifier validation des champs, gestion des doublons, confirmation, ou règles d'annulation ; sans spécification, l'IA choisit au hasard une interprétation.

C'est pourquoi les outils de développement piloté par les spécifications insistent sur le fait de décrire le **quoi** plutôt que le **comment**. Cette séparation réduit l'ambiguïté et rend la génération de code plus fiable.

- **Quoi** : résultat attendu, règles, contraintes, cas limites.
- **Pourquoi** : objectif produit, valeur utilisateur, priorité.
- **Comment** : choix techniques, laissé pour l'étape d'implémentation.

## Le rôle des documents vivants

Dans cette logique, la spécification n'est pas un document figé : elle évolue avec le projet. Plusieurs sources décrivent le développement piloté par les spécifications comme un processus où les artefacts servent à guider la génération, la validation et l'ajustement du travail de l'IA.

Cela est important parce qu'un projet logiciel change souvent en cours de route. Si le code est écrit sans support structuré, chaque changement devient une reprise manuelle plus coûteuse, alors qu'une spécification mise à jour garde le cap du système.

## Exemple concret de différence

Supposons une demande : “ajouter un panier d'achat” . Sans spécification, un agent peut produire une interface simple, mais oublier la persistance, les quantités, les remises ou la suppression d'article. Avec une spécification claire, ces règles apparaissent avant le code et deviennent vérifiables.

Autre exemple : “améliorer la connexion”. Le code seul peut ajouter un formulaire plus propre, mais une spécification peut préciser la gestion des erreurs, le verrouillage après plusieurs échecs et le message exact à afficher. C'est cette précision qui transforme une idée floue en comportement attendu.

## Ce que Spec Kit apporte à cette logique

Spec Kit formalise ce passage du flou à la clarté en organisant le travail autour d'artefacts structurés. La documentation officielle présente un flux prêt à l'emploi où l'on décrit d'abord l'intention, puis on affine en étapes successives, au lieu de demander directement du code complet à une IA.

Cette méthode rend le développement plus prévisible, car l'IA ne fabrique pas seulement des lignes de code : elle suit un cadre qui force à expliciter les exigences avant l'implémentation. C'est précisément pour cela que le **code seul** ne suffit plus dans un développement assisté par IA.

## 2. Spécifications exécutables qui guident l'IA

### Des spécifications exécutables

Une spécification exécutable est un document assez précis pour servir directement de guide à un agent IA, au lieu d'être une simple note de cadrage. Spec-Kit présente ce principe comme un moyen de transformer les intentions en systèmes fonctionnels, avec un flux **Spec** → **Plan** → **Tasks** → **Implement**.

L'idée clé est simple : la spécification ne décrit pas seulement *ce que l'on veut*, elle devient une contrainte de travail pour l'IA. GitHub explique que les *Executable Specifications* doivent être suffisamment précises, complètes et non ambiguës pour générer un système opérationnel.

### Pourquoi l'IA a besoin d'un cadre

Un agent IA peut produire du code rapidement, mais il improvise si les règles sont floues. Spec-Kit répond à ce problème en imposant des artefacts structurés, afin que l'agent suive un chemin défini plutôt qu'un prompt ad hoc.

Dans ce modèle, la qualité vient de la clarté du document source. Le résultat attendu est une réduction de l'écart entre l'intention et l'implémentation, car la spécification sert de référence commune pour le plan, les tâches et le code.

Une bonne spécification n'explique pas seulement le besoin : elle limite les interprétations possibles.

### Ce que contient une spécification utile

Une spécification utile décrit le *quoi* et le *pourquoi*, sans détailler immédiatement la technique. Les guides sur Spec-Kit recommandent de formuler les besoins métier, les comportements attendus et les critères d'acceptation avant de parler d'architecture.

On peut la lire comme un contrat de comportement. Par exemple :

- **Objectif** : permettre à un utilisateur de réserver un créneau.
- **Règle** : un créneau déjà pris ne doit pas être proposé.
- **Acceptation** : lorsque le créneau est occupé, l'interface affiche une alternative.

Cette structure aide l'IA à produire un code cohérent, parce qu'elle relie chaque fonctionnalité à une attente vérifiable.

## Comment Spec-Kit la rend opératoire

Spec-Kit fournit une CLI `specify` et des commandes qui organisent le travail de l'IA dans un ordre répétable. La documentation officielle décrit un processus prêt à l'emploi où la spécification est d'abord écrite, puis transformée en plan, en tâches, puis en implémentation.

Le point important est que la spécification n'est pas figée comme un texte décoratif. Elle alimente les étapes suivantes, et chaque étape reste traçable vers le document initial.

Exemple de séquence :

1. Décrire le besoin dans la spécification.
2. Transformer ce besoin en plan technique.
3. Découper le plan en tâches.
4. Faire générer le code par l'agent.

## Ce que cela change concrètement

Avec Spec-Kit, l'IA n'est plus seulement un générateur de fragments de code ; elle devient un exécutant guidé par des artefacts structurés. Plusieurs sources décrivent cette approche comme un développement piloté par spécification où les documents sont traités comme des éléments de travail de premier niveau.

En pratique, cela améliore la lisibilité des décisions, la vérification des écarts et la stabilité du résultat final. Pour un débutant, l'avantage le plus visible est clair : au lieu de demander "écris-moi l'application", on apprend à écrire une spécification que l'IA peut suivre sans deviner.

### 3. Cycle Spec Plan Tasks Implement expliqué

#### Le cycle en 4 étapes

Le flux **Spec** → **Plan** → **Tasks** → **Implement** est la colonne vertébrale de Spec-Kit : il transforme une idée floue en code traçable, étape par étape. Chaque phase réduit l'incertitude avant de passer à la suivante, ce qui limite les retours arrière coûteux

**Spec** décrit le *quoi* : les besoins, les objectifs et les critères d'acceptation vérifiables. **Plan** décrit le *comment* : architecture, choix techniques et structure des composants. **Tasks** découpe le plan en actions exécutables. **Implement** exécute ces actions une par une

#### Ce que produit chaque phase

La phase **Spec** crée une base claire avec des exigences métier et des critères testables, souvent rédigés de façon à pouvoir être validés sans ambiguïté. La phase **Plan** traduit cette intention en décisions techniques concrètes, comme les modèles de données ou les contrats d'API

La phase **Tasks** transforme le design en **liste ordonnée de tâches atomiques**. Une tâche atomique est une unité de travail petite, précise et vérifiable, souvent liée à un seul objectif de code. La phase **Implement** applique ces tâches en gardant le code aligné sur les artefacts précédents

#### Pourquoi ce découpage aide l'IA

Ce cycle empêche l'IA de “deviner” la solution trop tôt. Au lieu de produire du code à partir d'un simple prompt, elle suit un contexte structuré et stable. Spec-Kit insiste sur des validations humaines explicites entre les étapes, afin de confirmer que chaque passage reste cohérent

Cette méthode est particulièrement utile quand une fonctionnalité a plusieurs contraintes. Par exemple, si l'on demande “ajouter un formulaire de paiement”, le **Spec** fixe les règles métier, le **Plan** choisit la stratégie technique, les **Tasks** découpent le travail, puis **Implement** construit seulement ce qui a été prévu

#### Comment lire le passage de Plan à Tasks

Le passage de **Plan** à **Tasks** est souvent le moment le plus important. Le plan contient les choix structurants, mais il reste encore abstrait. Les tâches, elles, doivent être directement actionnables, testables et dans un ordre logique de réalisation

Dans plusieurs guides Spec-Kit, les tâches sont décrites comme un *checklist* dépendante du



plan, avec traçabilité vers la spec. Cela permet de savoir quelle partie du code correspond à quelle exigence, ce qui facilite la vérification et la maintenance future

## Exemple simple d'enchaînement

1. **Spec** : “L'utilisateur doit pouvoir réinitialiser son mot de passe avec un email valide.”
2. **Plan** : choisir un service d'envoi d'email, définir les routes et le stockage temporaire du token.
3. **Tasks** : créer la route, valider l'email, générer le token, écrire les tests.
4. **Implement** : coder chaque tâche dans l'ordre, puis vérifier que les critères d'acceptation sont satisfaits

Ce fonctionnement est dit **linéaire** dans la documentation Spec-Kit : chaque phase nourrit la suivante, et l'implémentation ne commence qu'une fois les tâches validées. C'est cette discipline qui rend le développement piloté par spécifications robuste et lisible pour l'IA comme pour l'humain

## 4. Prompts ad hoc contre contexte structuré

### Pourquoi le prompt ad hoc crée de la friction

Un **prompt ad hoc** est une consigne écrite “sur le moment”, sans cadre stable ni vocabulaire partagé. En développement assisté par IA, cela produit souvent des réponses inégales, parce que chaque demande dépend de l’humeur du contexte immédiat, pas d’une base explicite.

La logique formelle en IA sert justement à exprimer le raisonnement de façon précise et vérifiable, avec des règles et des faits clairement séparés.pdf). Dans une logique pilotée par spécifications, on cherche le même effet : réduire l’ambiguïté pour guider le système de manière reproductible.

### Ce que change un contexte structuré

Un **contexte structuré** organise les informations avant de lancer l’IA : objectif, contraintes, définitions, critères d’acceptation, exemples attendus. Cette approche ressemble à une base de connaissances où l’on injecte des faits utilisables par un moteur d’inférence.

Au lieu de dire “fais une API propre”, on fournit des éléments stables comme :

- le périmètre fonctionnel
- les règles métier
- les formats de sortie
- les limites techniques
- les cas d’erreur attendus

Cette structuration améliore la réutilisabilité et la tolérance aux changements, deux critères importants pour des systèmes logiques et explicites.

### Pourquoi Spec-Kit favorise ce modèle

Spec-Kit s’inscrit dans une logique où la **spécification** précède l’implémentation : on formalise d’abord ce que le logiciel doit faire, puis l’IA génère à partir de cette base. Cette méthode évite de laisser l’agent improviser à partir d’ordres courts et isolés, comme dans un prompt ad hoc.

Dans les approches fondées sur règles, les experts formulent souvent des instructions du type “si conditions alors actions”, évaluées ensuite sur des données structurées. C’est exactement l’idée utile ici : transformer une demande vague en consignes exploitables et contrôlables.

## Exemple simple de différence

Avec un prompt ad hoc :

Crée un formulaire de connexion sécurisé.

Avec un contexte structuré :

Objectif : authentification utilisateur.

Contraintes : email obligatoire, mot de passe masqué, message d'erreur  
↪ générique.

Sortie attendue : composant web avec validation côté client.

Cas limites : email vide, mot de passe vide, identifiants invalides.

Le second cas donne à l'IA des repères comparables à des faits et règles dans une base de connaissances. Le résultat est généralement plus cohérent, car le modèle ne doit pas deviner l'intention implicite.

## Comment raisonner au quotidien avec cette méthode

Pour adopter ce style, il faut remplacer la logique “je demande vite” par la logique “je spécifie clairement”. Le contexte structuré agit comme un contrat de lecture entre humain et IA, ce qui limite les interprétations contradictoires.

Utilise toujours :

- un **objectif mesurable**
- des **contraintes explicites**
- des **exemples concrets**
- des **critères de validation**
- un **format de sortie imposé**

Plus le contexte est explicite, moins l'IA doit interpréter ; plus elle interprète, plus le risque d'écart augmente.

Cette discipline rejoint l'idée centrale de l'IA logique : raisonner à partir de règles et de faits formulés proprement, plutôt qu'à partir d'instructions floues.

## 5. Historique et version 1.0 communautaire

### Origine et montée en maturité

Spec Kit est né comme un **toolkit open source** destiné au *Spec-Driven Development*, c'est-à-dire une façon de construire du logiciel à partir de spécifications explicites plutôt qu'à partir d'instructions vagues. Son dépôt officiel indique qu'un an après le premier commit, le projet avait atteint **1.0.0**, présenté non comme une fin, mais comme un signe de cohérence et d'adoption large.

Cette trajectoire est visible dans l'historique des versions : la série **1.0.x** apparaît avec **1.0.0** puis **1.0.1**, **1.0.2**, **1.0.3** et **1.0.4**. La page des releases montre aussi que **1.0.0** est datée du **21 août 2026**, suivie rapidement d'itérations correctives et documentaires.

### Ce que signifie la version 1.0

La mention **1.0** a ici une valeur importante : elle signale une première version jugée stable et suffisamment structurée pour être utilisée sérieusement. Le message du dépôt précise toutefois que cette stabilisation ne veut pas dire que le projet est “terminé” ou figé, mais qu'il a atteint une forme claire, utile et façonnée par de nombreux contributeurs.

En pratique, cela veut dire que les concepts centraux, les commandes principales et l'organisation du projet ont été suffisamment consolidés pour servir de base à l'écosystème. Cette étape marque souvent le passage d'un outil expérimental à un outil réellement adoptable par des équipes.

### Une version 1.0 portée par la communauté

La documentation communautaire montre que Spec Kit ne se limite pas au noyau maintenu par GitHub. Elle recense **157 extensions communautaires**, **90+ auteurs** et **33 presets**, ce qui illustre une croissance rapide de l'écosystème autour de la version 1.0.

La communauté ajoute des composants concrets :

- des **extensions**, qui étendent les capacités du système ;
- des **presets**, qui changent le comportement par défaut sans modifier l'outillage ;
- des **bundles**, qui regroupent plusieurs éléments pour un usage ciblé ;
- des projets compagnons, qui adaptent Spec Kit à des contextes spécifiques.

## Pourquoi parler de “communautaire”

Le terme **communautaire** est essentiel ici, car la version 1.0 n’est pas seulement un jalon technique interne. La documentation indique que les extensions et presets sont **contribués par la communauté**, puis intégrés dans des catalogues publics qui enrichissent l’expérience de Spec-Driven Development.

Un exemple simple : un preset communautaire peut **remplacer des templates, des commandes ou du vocabulaire** sans toucher au moteur principal. Cela permet à une équipe de garder la logique de Spec Kit tout en l’adaptant à son métier, comme un atelier qui conserve ses outils mais change ses gabarits.

## Lecture pratique de l’historique

Le passage à **1.0.0** s’accompagne d’un ensemble de correctifs et d’ajouts visibles dans les notes de publication : amélioration des presets, corrections du bundler, nouvelles extensions communautaires et ajustements des workflows. Ces changements montrent qu’une version 1.0 peut rester très active tout en gardant une base stable.

Pour un débutant, le bon réflexe est de lire cette histoire comme une progression en trois temps :

- **expérimentation** : le projet se structure ;
- **stabilisation** : la version 1.0 formalise la base ;
- **extension** : la communauté élargit les usages autour du noyau.

Le dépôt et les notes de version montrent ainsi que Spec Kit 1.0 est moins un point d’arrêt qu’un **socle communautaire** pour le développement logiciel guidé par spécifications.

## 6. Agents compatibles et intégrations disponibles

### Compatibilité des agents

Spec Kit est conçu pour fonctionner avec **plus de 30 agents de codage IA** et couvre à la fois les outils *CLI* et les assistants intégrés aux IDE. Cette compatibilité large permet de garder la même logique de travail, même si vous changez d'environnement ou d'assistant.

Un **agent compatible** est simplement un assistant capable d'exécuter les fichiers, commandes ou prompts générés par Spec Kit. En pratique, Spec Kit adapte ses sorties aux conventions de chaque outil : dossier dédié, format Markdown ou TOML, et parfois commande de lancement spécifique.

### Types d'intégration

On distingue surtout deux familles. Les agents **CLI** s'exécutent dans le terminal, comme **Claude Code**, **Gemini CLI**, **Qwen Code**, **Codex CLI**, **opencode** ou **Amazon Q Developer CLI**. Les agents **IDE-based** vivent dans l'éditeur, comme **GitHub Copilot**, **Cursor**, **Windsurf**, **Zed** ou **Roo Code**.

L'idée clé : Spec Kit ne force pas un seul assistant. Il normalise la structure des instructions pour que plusieurs agents puissent lire le même projet sans réécriture complète.

### Exemples concrets de prise en charge

Quelques intégrations bien documentées montrent cette logique : **Claude Code** installe des skills dans `.claude/skills` ; **Devin for Terminal** utilise `.devin/skills/` ; **Zed** et **Docker Agent** partagent `.agents/skills/` ; **RovoDev** génère `.rovo.dev/skills/`, des wrappers de prompts et `prompts.yml`.

D'autres agents suivent des conventions propres : **GitHub Copilot** s'appuie sur `.github/agents/`, **Cursor** sur `.cursor/commands/`, **Windsurf** sur `.windsurf/workflows/`, **Qwen Code** sur `.qwen/commands/`, et **Amp** sur `.agents/commands/`.

### Ce que cela change pour l'utilisateur

L'intérêt principal est la **portabilité**. Un même projet Spec Kit peut être partagé entre plusieurs équipes, chacune avec son assistant favori, sans changer la logique métier des spécifications. Cela réduit les frictions quand un développeur travaille dans VS Code, tandis qu'un autre utilise le terminal.

Cela rend aussi les workflows plus lisibles. Par exemple, un agent peut exécuter `/speckit-implement` ou une commande équivalente, tandis qu'un autre lit directement les commandes ou prompts déposés dans son dossier natif. L'effet est similaire à un adaptateur universel .

## À retenir pour bien choisir un agent

- Vérifiez si votre agent est **CLI** ou **IDE-based**.
- Regardez le **dossier cible** utilisé par l'intégration, car c'est lui qui reçoit les instructions générées.
- Confirmez le **format** attendu : Markdown, TOML ou Skill.
- Si vous travaillez en équipe, privilégiez un agent déjà listé comme **stable** dans la registry officielle.

Cette approche est utile pour débiter avec un agent simple comme **Claude Code**, **Gemini CLI** ou **GitHub Copilot**, puis élargir vers d'autres intégrations compatibles sans modifier la méthode de travail.

## 7. Presets extensions workflows et bundles

### Presets : personnaliser sans forker

Les **presets** servent à modifier le comportement existant de Spec Kit sans changer le cœur de l'outil. Ils remplacent ou ajustent des *templates*, des commandes et la terminologie, ce qui permet d'adapter le vocabulaire, les formats de sortie ou des standards internes .

Un preset est particulièrement utile quand une équipe veut garder la même base de travail, mais imposer ses propres conventions. Par exemple, un preset peut faire produire des spécifications plus strictes, des plans plus verbeux, ou un langage métier spécifique.

### Extensions : ajouter de nouvelles capacités

Les **extensions** ne changent pas seulement le style : elles ajoutent de nouvelles capacités à Spec Kit. Elles peuvent fournir des commandes spécifiques au domaine, des intégrations avec des outils externes, ou des *quality gates* qui valident le travail avant d'avancer .

En pratique, une extension est le bon choix si vous avez besoin d'une fonctionnalité qui n'existait pas. Par exemple, un workflow de revue de code autonome, une synchronisation avec Jira, ou une intégration de contrôle sécurité sont des cas typiques évoqués par la documentation communautaire.

### Workflow : enchaîner les étapes de façon répétable

Les **workflows** automatisent des séquences multi-étapes de développement piloté par spécifications. Ils peuvent enchaîner des commandes, des prompts, des étapes shell et des validations humaines, avec logique conditionnelle, boucles, fan-out et fan-in, ainsi que la reprise après interruption.

Cela signifie qu'un workflow n'est pas juste une liste de tâches. C'est une orchestration, comme un circuit de production : chaque étape prépare la suivante, et l'ensemble reste reproductible. Pour une équipe, cela réduit les variations entre deux exécutions du même processus.

### Bundles : emballer des composants prêts à installer

Un **bundle** regroupe des extensions, des presets, des workflows et des steps dans une unité versionnée et installable en une seule fois .

La documentation insiste sur un point important : un bundle **n'ajoute pas de logique runtime propre**. Il joue un rôle de composition et de distribution, avec *version pinning*,



vérification des conflits et traçabilité de provenance pour faciliter les mises à jour et les suppressions propres.

## Comment les utiliser ensemble

Le modèle mental le plus simple est celui-ci : **les extensions ajoutent, les presets transforment, les workflows orchestrent, et les bundles distribuent l'ensemble.**

Exemples concrets :

- un preset peut imposer une terminologie de conformité dans les artefacts produits ;
- une extension peut ajouter une commande de validation métier ;
- un workflow peut faire passer un document par plusieurs contrôles ;
- un bundle peut installer tout cela pour un rôle donné, comme développeur, analyste ou responsable sécurité.

Astuce pratique : si votre besoin est d'*adapter* un comportement existant, partez d'un preset ; si vous devez *créer* une capacité nouvelle, partez d'une extension.

## Priorité et empilement

Les presets sont **empilables** et ordonnés par priorité, ce qui permet de superposer plusieurs couches de personnalisation sans modifier les fichiers de base.

Cette logique est utile quand une organisation veut combiner des règles globales, puis des ajustements d'équipe, puis des adaptations locales. Le résultat est une personnalisation fine, mais contrôlée, qui reste compatible avec l'écosystème Spec Kit.

## 8. Cas d’usage logiciels et au-delà

### Cas d’usage logiciels

Spec-Kit sert d’abord à **transformer une idée logicielle en livrables précis**: spécification, plan, tâches, puis implémentation. GitHub le présente comme un toolkit open source pour le développement piloté par les spécifications, avec une séquence type `/constitution`, `/specify`, `/plan`, `/tasks`, `/implement`.

Dans un projet logiciel, cela aide surtout quand l’équipe veut éviter les ambiguïtés. Par exemple, pour une application de tri de photos, Xebia montre une démarche où l’on définit les principes du projet, rédige la spec, clarifie les cas limites, choisit la pile technique, puis génère la liste de tâches avant de coder.

- **Greenfield** : créer un produit de zéro avec une base claire.
- **Évolution d’existant** : ajouter une fonctionnalité sans casser les conventions déjà en place.
- **Feature complexe** : découper un besoin en étapes vérifiables.

### Au-delà du code

Spec-Kit ne se limite pas aux applications “classiques”. Un cas rapporté par Carbon concerne un **ETL** où les équipes fonctionnelles rédigent les besoins en langage naturel, puis les traduisent en fichiers structurés avant génération des spécifications et du code. Cela montre que l’outil convient aussi aux flux de données, à l’intégration et à l’automatisation .

Microsoft décrit aussi un usage en environnement d’entreprise avec des contraintes explicites de **sécurité**, **performance** et **conformité**, encodées dans une “constitution” de projet pour guider les plans générés. Autrement dit, Spec-Kit peut servir à formaliser des règles métier ou organisationnelles, pas seulement à produire des écrans ou des API.

### Cas d’équipe et de contribution

L’intérêt principal apparaît quand plusieurs personnes travaillent sur le même périmètre. Spec-Kit aide à faire valider la spec avant le code, ce qui réduit les retours tardifs et aligne les rôles fonctionnels et techniques. Dans un projet legacy testé par XB Software, une tâche d’une semaine a été réalisée en deux fois moins de temps, avec détection de manques dans les exigences.

Ce fonctionnement est utile pour :

- les équipes produit qui veulent documenter avant d’implémenter;
- les équipes de maintenance qui ajoutent une fonctionnalité à un code existant;

- les projets open source qui examinent d'abord la spec d'une contribution avant d'accepter le code.

## Quand Spec-Kit est le plus pertinent

Spec-Kit donne ses meilleurs résultats sur des tâches **bien décrites** et de taille moyenne à grande. XB Software indique explicitement qu'il fonctionne mieux pour des tâches larges et clairement définies. Cela en fait un bon choix pour un module métier, une intégration d'API, un workflow interne ou une migration guidée par contraintes.

L'idée centrale est simple : plus le besoin est structuré, plus l'IA peut produire un résultat utile sans deviner.

## Exemples concrets

Un portail peut recevoir une fonctionnalité de feedback: bouton, formulaire, pièce jointe optionnelle, notification à l'équipe support, et intégration avec un outil d'analytics comme Amplitude. Spec-Kit force alors à écrire les règles avant le code, ce qui évite de mélanger interface, suivi et logique métier.

Autre exemple: une équipe qui construit une application interne de gestion documentaire peut utiliser une spec pour décrire les rôles, les droits d'accès et les validations. Le résultat attendu n'est pas seulement un programme, mais un comportement reproductible .

## 9. Prérequis uv sur Windows macOS Linux

### Prérequis sur Windows, macOS et Linux

uv est un outil autonome : il peut s'installer sans Python préalable, ce qui le rend très pratique pour démarrer vite . Les documentations d'Astral indiquent qu'il est pris en charge en **Tier 1** sur macOS Apple Silicon, macOS x86\_64, Linux x86\_64 et Windows x86\_64, avec un support de build plus large pour d'autres architectures Linux et Windows arm64.

Pour ce chapitre, l'idée est simple : avant d'utiliser **Specify-CLI**, il faut que uv soit disponible dans le terminal, puis vérifié avec sa version.

### Ce qu'il faut vérifier avant l'installation

Sur **Windows**, la version minimale prise en charge est **Windows 10** et **Windows Server 2016**. Sur **macOS**, uv supporte **macOS 13+ (Ventura)**, et il fonctionne aussi sur macOS 12, mais avec une dépendance supplémentaire à **realpath**. Sur **Linux**, les variantes x86\_64 sont pleinement supportées, et d'autres architectures sont aussi prévues selon le niveau de support annoncé par Astral.

- Vérifiez votre système avant d'installer.
- Utilisez un terminal avec droits adaptés.
- Gardez à l'esprit que uv peut être installé *sans* gestionnaire Python existant.

### Installation recommandée selon le système

La méthode officielle la plus directe consiste à utiliser le script d'installation Astral sur **macOS** et **Linux**.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Sur **Windows**, l'installation recommandée se fait via PowerShell :

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1  
↪ | iex"
```

Sur macOS, uv est aussi disponible via **Homebrew**.

```
brew install uv
```

### Contrôle après installation

Après l'installation, il faut confirmer que le binaire est bien accessible depuis le **PATH**, c'est-à-dire la liste des dossiers que le terminal parcourt pour trouver une commande.

```
uv --version
```

Si la commande répond avec une version, l'installation est correcte. Les guides consultés recommandent aussi de vérifier que le binaire a bien été ajouté au **PATH**, surtout après une installation par script sur Linux ou macOS.

## Cas pratiques utiles pour préparer Specify-CLI

Une fois **uv** installé, vous disposez d'un environnement prêt pour les outils Python modernes utilisés dans les flux de travail basés sur spécifications. **uv** sert alors de base pour lancer des commandes, gérer des environnements isolés et installer des outils sans alourdir la machine.

- **Exemple Windows** : ouvrir PowerShell, lancer le script officiel, puis exécuter **uv --version**.
- **Exemple macOS** : installer via **brew install uv**, puis vérifier la version dans le terminal.
- **Exemple Linux** : utiliser le script officiel Astral, puis contrôler l'accès à la commande.

Si **uv** n'est pas reconnu, le problème vient le plus souvent du **PATH** ou d'une installation incomplète ; dans ce cas, il faut rouvrir le terminal après l'installation et relancer la vérification.

## 10. Installation via dépôt Git et PyPI

### Installation depuis GitHub

L'installation **depuis le dépôt Git** est la voie la plus directe pour obtenir une version précise de l'outil, car elle pointe vers un *tag* de release. La documentation officielle recommande cette méthode comme chemin source, avec `uv` pour installer `specify-cli` depuis `github/spec-kit.git` en conservant le préfixe `v` du tag, par exemple `v0.12.11`.

Le principe est simple : vous installez la CLI à partir du code publié par le dépôt officiel, puis vous pouvez garder cette version figée pour des déploiements reproductibles.

La commande type est la suivante :

```
uv tool install specify-cli --from  
↪ git+https://github.com/github/spec-kit.git@vX.Y.Z
```

Remplacez `vX.Y.Z` par le tag exact souhaité, comme `v1.0.4`. Cette approche est utile si vous voulez une version déterminée, stable et identique d'une machine à l'autre.

### Pourquoi choisir Git plutôt que l'installation standard

Installer depuis GitHub est pertinent quand vous voulez contrôler précisément la version utilisée. GitHub indique que cette méthode est recommandée pour installer depuis la source, alors que l'installation PyPI est pensée pour utiliser le paquet publié avec vos outils Python habituels.

Vous pouvez aussi forcer une réinstallation d'une version donnée avec `uv`, ce qui est pratique pour mettre à jour la CLI sans modifier un projet existant. GitHub documente par exemple l'usage de `--force` avec un tag précis.

### Installation depuis PyPI

Spec Kit est également publié sur **PyPI** sous le nom `specify-cli`. La documentation officielle indique que vous pouvez l'installer avec votre gestionnaire Python préféré, sans utiliser l'URL Git.

Les commandes courantes sont :

```
uv tool install specify-cli  
pipx install specify-cli  
pip install specify-cli
```

La documentation met `uv` en avant comme option recommandée, mais `pipx` et `pip` restent pris en charge. Cela rend l'installation plus simple si vous utilisez déjà un environnement Python classique.

## Versionner l'installation pour éviter les écarts

Pour un résultat reproductible, la documentation conseille de **geler la version exacte**. Cela signifie installer une révision précise du paquet, afin d'éviter qu'une mise à jour modifie le comportement de votre CLI au fil du temps.

Exemple avec une version épinglée :

```
uv tool install specify-cli==0.12.11
pipx install specify-cli==0.12.11
pip install specify-cli==0.12.11
```

Cette pratique est particulièrement utile en équipe, car tous les membres exécutent la même version de l'outil.

## Cas particuliers avec index privé ou miroir

Certains environnements d'entreprise utilisent un index Python interne, un miroir ou un proxy. La documentation PyPI précise alors qu'il faut fournir un **index alternatif** via une variable d'environnement ou un argument de commande.

Exemples pratiques :

```
UV_DEFAULT_INDEX=https://your-index.example.com/pypi/simple/ uv tool install
  ↳ specify-cli
uv tool install --default-index https://your-index.example.com/pypi/simple/
  ↳ specify-cli
pipx install specify-cli --index-url
  ↳ https://your-index.example.com/pypi/simple/
pip install specify-cli --index-url
  ↳ https://your-index.example.com/pypi/simple/
```

Cela permet d'installer `specify-cli` même quand l'accès au dépôt public est limité par la politique réseau.

## 11. Vérifier mise à jour et dépannage

### Vérifier l'état de l'outil

Pour confirmer que **Specify-CLI** est bien installé et répond, exécute `specify check` dans ton terminal. La documentation officielle indique que cette commande vérifie le bon fonctionnement de la CLI et affiche les outils installés, tandis que `specify self check` compare la version installée à la dernière version publiée sur GitHub.

```
specify check
specify self check
```

`specify check` sert donc de **contrôle local**, alors que `specify self check` sert de **contrôle de mise à jour**. La distinction est utile : le premier vérifie que la commande marche, le second vérifie si une version plus récente existe.

### Mettre à jour la CLI

La méthode recommandée pour passer à la dernière version stable est `specify self upgrade`. La documentation précise que cette commande met à jour l'outil *en place* et détecte automatiquement si l'installation vient de `uv tool` ou de `pipx`.

```
specify self upgrade
```

Si tu veux viser une version précise, utilise `specify self upgrade --tag vX.Y.Z[suffix]`. GitHub précise que cette option permet de verrouiller une version donnée, avec des suffixes autorisés comme `dev`, `alpha`, `beta`, `rc` ou des métadonnées de build.

```
specify self upgrade --tag vX.Y.Z
```

### Reprendre la main si la mise à jour automatique ne marche pas

Quand `specify self upgrade` n'est pas disponible, la documentation propose un **fallback manuel**. Pour une installation via `uv`, la commande de réinstallation forcée est `uv tool install specify-cli --force --from git+https://github.com/github/spec-kit.git@vX.Y.Z`; pour `pipx`, c'est `pipx install --force git+https://github.com/github/spec-kit.git@vX.Y.Z`.

```
uv tool install specify-cli --force --from
  ↪ git+https://github.com/github/spec-kit.git@vX.Y.Z
pipx install --force git+https://github.com/github/spec-kit.git@vX.Y.Z
```

Cette approche est utile quand tu veux **contrôler exactement** la version installée, ou quand la commande d'auto-mise à jour est absente sur une ancienne installation.



## Vérifier le résultat après mise à jour

Après l'upgrade, relance `specify check` pour confirmer que la CLI répond correctement. La documentation recommande aussi `specify self check` pour confirmer la version installée par rapport à la dernière release GitHub.

```
specify check
specify self check
```

Un bon réflexe consiste à vérifier aussi la version affichée par le binaire avec `specify version`. La documentation d'installation explique que cette commande confirme simplement que `specify` est bien accessible dans le terminal et renvoie la version attendue.

```
specify version
```

## Dépannage des problèmes courants

Si la commande n'est pas reconnue, le problème vient souvent du **PATH**, c'est-à-dire la liste des dossiers où le terminal cherche les exécutables. Un guide communautaire sur Spec Kit indique que `uv tool update-shell` peut être nécessaire, puis il faut rouvrir le terminal avant de réessayer.

Si `specify` est introuvable, vérifie d'abord l'accès au shell, puis teste à nouveau la commande.

En cas de doute sur la syntaxe ou la commande exacte, la documentation et les guides d'usage citent surtout `specify check`, `specify self check` et `specify self upgrade`; ils constituent le trio de base pour diagnostiquer une installation et la remettre à jour sans casser l'environnement.

## 12. Préparer un agent IA de codage

### Préparer l'agent dans le bon dossier

Avant de lancer un agent de codage, placez-vous dans le répertoire du projet généré par `specify init`. GitHub Spec Kit indique explicitement de lancer l'agent *dans le dossier du projet* pour que les commandes spécifiques au workflow soient disponibles.

Le but est simple : l'agent doit lire la structure créée par `specify`, puis exécuter les commandes de développement guidé par spécifications. Cette préparation évite qu'il code "à l'aveugle" et l'aligne sur le dossier courant .

L'agent n'est pas utilisé comme un simple générateur de code, mais comme un exécutant d'un cycle structuré : spécifier, planifier, découper, implémenter, converger.

### Choisir l'intégration d'agent adaptée

La commande d'initialisation supporte une intégration précise, par exemple `copilot` ou `claude`, afin de générer les bons fichiers et la bonne structure de commandes. La documentation précise que `specify init` configure automatiquement le projet selon l'agent choisi.

Exemple de préparation pour GitHub Copilot :

```
uv tool install specify-cli --from
↪ git+https://github.com/github/spec-kit.git@vX.Y.Z
specify init my-project --integration copilot
cd my-project
```

GitHub recommande de remplacer `vX.Y.Z` par le dernier tag de version publié.

### Définir la séquence de travail de l'agent

Une fois l'agent lancé dans le projet, Spec Kit propose une progression claire. La documentation liste six commandes de travail : `speckit-constitution`, `speckit-specify`, `speckit-plan`, `speckit-tasks`, `speckit-implement` et `speckit-converge`.

1. **Constitution** : fixer les principes du projet une seule fois.
2. **Spécification** : décrire ce qu'il faut construire.
3. **Plan** : définir la manière de le construire.
4. **Tâches** : découper le plan en actions concrètes.
5. **Implémentation** : générer le code.
6. **Convergence** : vérifier l'écart entre code, spec et plan.

Cette logique fonctionne comme un contrat de travail entre vous et l'agent : d'abord le cadre, ensuite l'exécution.

## Préparer le contexte que l'agent va consommer

Spec Kit installe des fichiers de commande et des structures de dossiers adaptés à l'agent choisi, afin que celui-ci dispose du contexte nécessaire sans configuration manuelle lourde. La documentation précise que cette génération est automatique lors de `specify init`.

En pratique, cela signifie que l'agent doit trouver :

- les commandes spécifiques à Spec Kit ;
- les artefacts de spec et de plan ;
- la structure attendue pour travailler de façon reproductible.

Si vous utilisez un agent CLI, la préparation consiste donc surtout à vérifier que le bon répertoire est ouvert et que les commandes ont bien été générées pour l'intégration choisie.

## Bon réflexe avant de coder avec l'IA

L'approche Spec Kit repose sur un développement *document-first* : l'agent suit une séquence guidée au lieu de produire directement du code. Des sources décrivent ce mode comme un flux structuré “spec → plan → implement” destiné à éviter le code improvisé.

Exemple d'usage utile :

```
specify init . --integration copilot
```

Cette commande initialise aussi bien un projet neuf qu'un dépôt existant, à condition de l'exécuter dans le bon dossier.

Gardez une règle simple : **préparer l'agent**, c'est lui donner le bon contexte, le bon dossier et le bon workflow avant toute génération de code.

## 13. Initialiser un projet avec intégration agent

### Lancer l'initialisation

Pour démarrer un projet Spec Kit avec un agent IA intégré, la commande de base est `specify init <nom-projet> --integration <clé-agent>`. GitHub indique par exemple `specify init my-project --integration copilot`, puis `cd my-project` pour entrer dans le dossier créé.

Cette étape **bootstrappe** le projet : elle crée la structure de dossiers, les modèles, les scripts et les fichiers de configuration nécessaires à l'agent choisi. En pratique, cela évite de tout configurer à la main, un peu comme un kit de montage qui prépare déjà les pièces utiles .

### Choisir l'intégration d'agent

L'option `--integration` sert à sélectionner l'agent de codage IA à utiliser, comme `copilot`, `claude` ou `gemin`. La documentation précise que Spec Kit installe les fichiers de commande et la structure adaptés à l'agent choisi, afin de lancer directement le flux Spec-Driven Development.

Une **intégration** relie Spec Kit à un agent IA précis en générant les bons fichiers et réglages pour lui.

Si vous démarrez un projet neuf, le choix de l'agent se fait dès `specify init`, ce qui évite une installation séparée ensuite. C'est la méthode recommandée par la documentation pour un projet "prêt à l'emploi" avec un agent actif dès le départ.

### Exemple concret de création

Un exemple simple ressemble à ceci :

```
specify init my-project --integration copilot
cd my-project
```

GitHub documente aussi la variante avec `claude`. Dans tous les cas, Spec Kit génère les éléments nécessaires pour que l'agent puisse fonctionner dans le projet, avec les bons fichiers de commandes et de contexte.

Pour un agent non listé, la documentation mentionne l'intégration **generic** comme solution de repli. Cela permet de garder une compatibilité minimale, même si l'agent ne dispose pas d'un connecteur dédié.

## Initialiser dans un dossier existant

Si le projet existe déjà, `specify init . --integration copilot` initialise Spec Kit dans le dossier courant. La documentation montre aussi `specify init --here --force --non-interactive --integration claude` pour forcer l’installation dans un dossier non vide.

- `.` ou `--here` signifie “utiliser le dossier courant”.
- `--force` autorise l’initialisation dans un répertoire déjà rempli.
- `--non-interactive` évite toute saisie manuelle pendant l’exécution.

Cette combinaison est utile quand on travaille dans un dépôt déjà créé ou quand on automatise la mise en place du projet sur une machine de développement .

## Points pratiques à retenir

La CLI vérifie aussi la disponibilité des agents IA basés sur ligne de commande sur la machine, avant de finaliser l’initialisation. La documentation précise qu’une seule intégration est active par projet à la fois, mais qu’on peut en changer plus tard.

Pour les environnements sans interaction, comme les pipelines CI ou certains harness IA, le mode `--non-interactive` est recommandé pour empêcher le blocage sur un sélecteur. Si le dossier cible n’est pas vide, `--force` permet d’éviter un arrêt prématuré lors du lancement.

## 14. Comprendre dossiers et fichiers générés

### Dossiers créés automatiquement

Quand Spec-Kit initialise un projet, il génère une arborescence pensée pour séparer les *intentions*, les *spécifications* et les *sorties prêtes à l'exécution*. Cette organisation aide l'IA à travailler de façon structurée, un peu comme un bureau avec des tiroirs dédiés .

Vous voyez généralement un dossier racine de projet, puis des sous-dossiers qui servent à stocker les éléments de conception, les documents de travail et parfois les artefacts techniques produits par l'outil. L'objectif est de garder le projet lisible dès le départ.

- **Dossier racine** : point d'entrée du projet.
- **Dossier de spécifications** : contenu décrivant ce que le logiciel doit faire.
- **Dossier d'artefacts** : fichiers générés à partir des spécifications, selon le flux de travail.

### Fichiers de spécification

Le cœur du projet repose sur des fichiers de spécification, c'est-à-dire des fichiers qui décrivent le comportement attendu avant toute implémentation. Dans une approche guidée par prompt, ces fichiers jouent le rôle de contrat fonctionnel entre vous et l'outil IA.

Ces fichiers sont utiles parce qu'ils rendent la demande explicite : fonctionnalités, règles métier, cas limites, et parfois critères d'acceptation. L'IA peut ensuite s'appuyer sur ce texte pour générer un code plus cohérent et mieux aligné avec l'objectif initial.

Pensez à la spécification comme à une *recette* : elle décrit le résultat attendu avant la cuisson, pas seulement les ingrédients.

### Fichiers de projet et métadonnées

Spec-Kit peut aussi créer des fichiers de configuration ou de métadonnées qui indiquent comment le projet doit être compris et traité. Ces fichiers servent à conserver des informations stables : nom du projet, structure attendue, ou paramètres liés au flux de génération.

Dans un projet IA, ces fichiers sont importants parce qu'ils évitent de répéter les mêmes consignes à chaque session. Ils facilitent aussi la reprise du travail par un autre membre de l'équipe ou par un agent IA différent, sans perdre le contexte.

- **Métadonnées** : informations sur le projet, pas sur le produit final.
- **Configuration** : paramètres qui orientent le comportement de l'outil.

- **Traçabilité** : capacité à retrouver pourquoi un fichier existe et comment il a été produit.

## Fichiers générés par l'outil

Après lecture des spécifications, Spec-Kit peut produire des fichiers dérivés, par exemple du code initial, des structures de dossier supplémentaires ou des squelettes de composants. Ces fichiers ne sont pas la source de vérité : ils découlent des spécifications et peuvent être régénérés si nécessaire.

Cette logique est pratique pour le prototypage rapide, car elle réduit le temps entre l'idée et une première version utilisable . Google Cloud décrit d'ailleurs un flux similaire où l'IA génère du code initial à partir d'un objectif exprimé en langage naturel.

## Lire l'arborescence sans se perdre

Pour comprendre rapidement un projet généré, commencez par repérer les fichiers qui décrivent *quoi faire*, puis ceux qui montrent *ce qui a été produit*. Cette lecture vous aide à distinguer le plan, la configuration et l'implémentation réelle.

- Repérez d'abord les fichiers de description fonctionnelle.
- Cherchez ensuite les fichiers de configuration.
- Terminez par les fichiers générés automatiquement pour vérifier s'ils correspondent bien à la spec.

Cette méthode est simple, mais très efficace quand on débute. Elle vous permet de savoir où modifier une règle, où ajuster un paramètre, et où intervenir dans le code sans casser la structure du projet .

## 15. Mode non interactif pour CI et automatisation

### Exécuter sans interaction

Le **mode non interactif** permet de lancer un agent IA depuis un script, un job CI ou une automatisation, puis de récupérer le résultat sans ouvrir de session conversationnelle. Pour Claude Code, l'entrée standard est `-p` ou `--print`, qui traite un seul prompt, affiche la réponse, puis quitte immédiatement.

En pratique, cela transforme l'agent en *outil de commande* : une requête, une sortie, fin d'exécution.

### Quand l'utiliser

Ce mode est adapté aux tâches **bornées** : vérifier un dépôt, résumer des tests, analyser un diff, produire un rapport ou générer un artefact de build. Les sources sur Codex décrivent le même principe avec `codex exec`, conçu pour les scripts, pipelines CI et tâches répétables.

- Vérifier un changelog avant publication.
- Résumer les échecs de tests dans un pipeline.
- Produire une revue de risque sur une modification ciblée.
- Générer une réponse JSON exploitable par un autre outil.

### Commande de base

Pour Claude Code, la forme la plus simple est :

```
claude -p "Dis bonjour"
```

Le flag `-p` lance une exécution **sans interface interactive**, ce qui évite d'attendre une saisie utilisateur. La documentation précise aussi que ce mode peut s'utiliser avec des options CLI supplémentaires selon le besoin.

Pour Codex, l'équivalent est :

```
codex exec "Review this repository and list the highest-risk issues."
```

Ici, `exec` désigne explicitement le mode non interactif, avec une sortie pensée pour l'automatisation.

### Intégration dans CI/CD

Dans un pipeline, l'objectif est de rendre la sortie **prévisible** et **machine-readable**. Les guides recommandent d'utiliser un format JSON ou stream JSON quand c'est disponible, afin



de chaîner le résultat avec `jq`, `grep` ou des scripts shell. Claude Code mentionne aussi l'usage de `--output-format json` ou `--output-format stream-json` pour la sortie exploitable par machine.

Un exemple de logique CI simple est :

```
- name: Analyser le diff
  run: claude -p "Résume les risques de cette modification"
```

Les sources montrent également des déploiements dans GitHub Actions avec gestion des secrets, ce qui confirme l'usage en environnement automatisé.

## Points de vigilance

Le mode non interactif doit rester **déterministe** autant que possible : prompt court, objectif précis, format de sortie attendu. Les guides sur Codex insistent sur des tâches bornées et sur la visibilité des échecs dans les logs de CI.

- Utiliser un prompt limité à une seule intention.
- Exposer clairement les erreurs dans les journaux.
- Réserver les accès sensibles à des secrets de pipeline.
- Éviter toute dépendance à une confirmation humaine pendant l'exécution.

Pour Claude Code, une source indique que `--dangerously-skip-permissions` devient nécessaire pour une exécution non interactive dans un pipeline, ce qui souligne que l'automatisation complète implique aussi une attention particulière aux autorisations.

## 16. Changer d'agent sans verrouillage

### Changer d'agent sans verrouillage

Changer d'agent sans verrouillage signifie **basculer vers un autre agent de codage** sans perdre la configuration déjà installée ni réécrire le projet. Dans Spec Kit, cette logique repose sur la commande `specify integration switch <key>`, qui change l'intégration active tout en gardant le projet cohérent.

### Ce que veut dire “sans verrouillage”

Le mot *verrouillage* désigne ici une situation où un seul agent reste imposé par la structure du projet. Spec Kit évite cela en permettant de **garder plusieurs intégrations installées** et de changer l'agent par défaut quand on en a besoin, au lieu de tout désinstaller et réinstaller. Cela permet de passer, par exemple, d'un agent orienté rédaction à un agent plus adapté à l'implémentation, sans casser le flux de travail .

### La commande à utiliser

La commande documentée est simple :

```
specify integration switch <key>
```

Le paramètre `<key>` correspond à la clé de l'intégration que vous voulez activer. Une source de la documentation explique que cette commande **modifie l'intégration par défaut sans en supprimer les autres**. Une autre ressource précise que, dans certaines versions ou présentations, le changement peut retirer l'intégration précédente du dossier actif, ce qui montre que le comportement exact dépend de l'édition ou de l'outil utilisé autour de Spec Kit.

### Quand l'utiliser

Utilisez ce changement quand vous voulez adapter l'agent à une tâche précise sans repartir de zéro. Par exemple, vous pouvez conserver vos spécifications en Markdown, puis basculer vers un autre exécuteur pour continuer le travail avec le même cadre de projet. C'est utile pour alterner entre plusieurs assistants de code selon la phase du développement : analyse, écriture, correction ou revue .

Un usage concret ressemble à ceci :

```
specify integration switch cursor
```

Après cette commande, l'intégration active devient celle associée à **cursor** pour les prochaines actions du projet.

## Bonnes pratiques pour éviter les erreurs

- Vérifiez d'abord les intégrations disponibles avec `specify init` ou la commande de gestion prévue par votre environnement, afin de choisir une clé valide.
- Préférez la commande officielle plutôt qu'une modification manuelle des dossiers d'agent ou du fichier `config.yaml`, car la documentation recommande d'éviter les edits directs pour préserver la synchronisation.
- Si vous travaillez dans un projet partagé, notez quel agent a été activé, pour que l'équipe comprenne le contexte de l'environnement .
- Si votre outil propose plusieurs agents installés en même temps, utilisez le commutateur prévu pour changer le défaut sans tout retirer.

## Ce qu'il faut retenir dans la pratique

Le principe central est de **changer l'agent actif, pas le projet**. Spec Kit permet cela parce qu'il sépare la spécification du moteur d'exécution : vos règles restent, l'agent change. C'est cette séparation qui rend le passage d'un agent à l'autre fluide, propre et réversible, sans verrouiller votre projet sur un seul assistant.

## 17. Rôle du fichier constitution du projet

### Ce que fait le fichier de constitution

Le fichier **constitution** est la référence centrale qui fixe les **principes non négociables** du projet. Dans Spec Kit, la commande `/speckit.constitution` sert à créer ou mettre à jour ce document de gouvernance, puis les autres étapes du flux s'y conforment.

Pensez-y comme à la **loi du projet** : elle dit ce qui est autorisé, obligatoire ou interdit, avant même d'écrire une spécification.

### Ce qu'il contient

Ce document rassemble les règles qui orientent toute décision technique. Les sources décrivent notamment des règles d'architecture, de qualité, de tests, de sécurité, de style de code et de pile technologique, avec des versions si nécessaire.

- **Architecture** : sens des dépendances, organisation des modules, séparation des responsabilités.
- **Qualité** : conventions de nommage, lisibilité, documentation, robustesse.
- **Tests** : exigences de tests unitaires ou d'intégration, couverture attendue, approche TDD éventuelle.
- **Sécurité** : validation des entrées, absence de secrets en dur, gestion des erreurs.

### Pourquoi il est essentiel

La constitution agit comme un **filtre de cohérence**. Elle permet à l'IA de générer des artefacts compatibles avec les règles du projet, et elle évite que les plans ou le code contredisent les décisions fondatrices. Plusieurs sources indiquent qu'elle est lue avant les autres commandes et qu'elle valide les livrables produits ensuite.

Cela la distingue d'un README : le README explique surtout **ce que fait** le projet, alors que la constitution définit **comment il doit être construit** et gouverné.

### Comment l'utiliser correctement

La bonne pratique consiste à rédiger des principes **clairs, testables et sans ambiguïté**. Les formulations du type *MUST* et *MUST NOT* sont recommandées, car elles réduisent les interprétations vagues et facilitent la vérification.

1. Définir les règles non négociables du projet.
2. Relire le fichier généré et l'ajuster selon le contexte réel.

3. Garder ce document cohérent avec les décisions d'architecture et de qualité.

## Exemple concret

Un principe de constitution peut dire : **“Toutes les fonctionnalités doivent être autonomes, indépendantes et testables.”** Cette règle, citée dans plusieurs explications de Spec Kit, sert de garde-fou pour que chaque évolution reste compatible avec la structure voulue.

Autre exemple utile : **“Les erreurs doivent être gérées explicitement, sans secrets codés en dur.”** Ce type de règle rend les attentes vérifiables et aide l'IA à produire un résultat plus fiable.

## 18. Rédiger principes testabilité et qualité

### Pourquoi écrire des principes de testabilité et de qualité

La **constitution** fixe les règles non négociables qui guident toute génération de code par l'IA. Dans Spec Kit, elle sert de référence durable pour éviter un code “acceptable” mais fragile, difficile à tester ou incohérent avec le projet.

La **testabilité** désigne la capacité d'un code à être vérifié facilement par des tests automatisés. La **qualité** regroupe des exigences concrètes comme la lisibilité, la simplicité, la gestion explicite des erreurs et la couverture de test minimale.

Une bonne constitution ne décrit pas seulement *ce qu'il faut faire* ; elle impose aussi *comment le faire proprement*.

### Ce que doivent contenir ces principes

Écrivez des règles courtes, vérifiables et sans ambiguïté. Par exemple, Spec Kit recommande de formuler des principes de projet avant les exigences détaillées, avec des attentes explicites sur les tests et les standards techniques.

Les principes les plus utiles concernent souvent :

- la **couverture de test** minimale ;
- l'**isolement** des effets de bord ;
- la **préférence pour du code simple** plutôt que “intelligent” ;
- la **gestion explicite des erreurs** ;
- la **traçabilité** entre exigence, test et code.

Un principe bien écrit doit pouvoir être contrôlé. Par exemple, “le code doit être lisible” est vague, alors que “chaque logique métier importante doit avoir des tests unitaires” est actionnable.

### Exemples de formulations efficaces

Voici des formulations adaptées à une constitution Spec Kit :

- Toute logique métier critique doit être couverte par des tests unitaires.
- Les comportements publics doivent disposer de tests d'intégration quand
  - ↪ ils traversent plusieurs composants.
- Les erreurs doivent être explicites et testées, jamais masquées
  - ↪ silencieusement.

- Les modules doivent rester petits et isolables pour faciliter le mocking.
- La couverture minimale des tests est de 80 % sur le code métier.

Ces principes sont cohérents avec les exemples de Spec Kit qui mettent en avant les tests, la qualité du code et les contraintes de conception comme éléments structurants de la constitution.

## Règles de rédaction à privilégier

Formulez vos principes avec des verbes d'action et des critères observables. Une bonne règle doit permettre à l'IA de décider si une proposition respecte ou non la constitution.

Pour renforcer la testabilité :

- imposez des **interfaces claires** ;
- limitez les dépendances cachées ;
- séparez la logique métier du code d'infrastructure ;
- demandez des **tests contractuels** pour les frontières du système.

Pour renforcer la qualité :

- imposez une **gestion d'erreurs explicite** ;
- exigez des noms compréhensibles ;
- interdisez les abstractions inutiles ;
- demandez une cohérence entre styles de code et comportements attendus.

## Exemple de mini-constitution orientée qualité

Le projet privilégie :

- un code testable plutôt qu'un code astucieux ;
- des erreurs explicites plutôt que des échecs silencieux ;
- des fonctions pures lorsque c'est possible ;
- des tests automatisés avant l'implémentation ;
- une structure modulaire pour faciliter la vérification.

Cette approche reflète l'idée centrale de Spec Kit : la spécification devient la source de vérité, et la constitution fixe le cadre qui empêche l'IA de s'en écarter.

## 19. Faire modifier la constitution par l'IA

### Modifier la constitution avec l'IA

La modification de la constitution consiste à demander à l'IA d'**adapter les principes du projet** quand ils doivent évoluer, sans réécrire tout le cadre. Spec-Kit prévoit un flux explicite : partir de `.specify/memory/constitution.md`, remplir les champs manquants, puis propager les changements aux modèles dépendants.

L'objectif n'est pas d'ajouter du texte "joli", mais de produire des règles **claires, testables et stables** que l'IA pourra appliquer sans ambiguïté.

### Quand lancer une modification

Utilise cette étape quand un besoin réel change, par exemple une nouvelle politique d'équipe, une contrainte technique, ou une leçon apprise après livraison. Spec-Kit recommande d'actualiser la constitution dans ces cas, avec un type de mise à jour adapté : ajout, révision ou extension.

- **Ajout** : nouveau principe de gouvernance.
- **Révision** : règle existante devenue inadaptée.
- **Extension** : précision d'une règle sans casser le cadre global.

Cette logique protège la cohérence du projet : la constitution reste la source des règles "WHAT/MUST", pas un document d'implémentation "HOW".

### Comment l'IA procède

L'agent charge d'abord la constitution existante et repère les tokens de type `[IDENTIFIANT_EN_CAPS]`. Il cherche ensuite les valeurs dans la conversation, le dépôt, ou les versions précédentes, puis remplace chaque placeholder par un texte concret.

Exemple simple :

```
[PROJECT_NAME] doit privilégier les tests automatiques avant toute  
↪ livraison.
```

peut devenir :

```
Le projet Atlas doit privilégier les tests automatiques avant toute  
↪ livraison.
```

L'IA ajuste aussi les métadonnées de gouvernance, comme la **date de ratification** et la **date de dernière modification** si la constitution a changé.



## Ce que l'IA doit vérifier avant d'écrire

Spec-Kit demande une validation de cohérence avant la sortie finale. L'agent doit contrôler qu'aucun placeholder n'a été oublié, que les règles restent compatibles avec les modèles liés, et que la mise à jour n'introduit pas de contradiction interne.

- Vérifier les principes modifiés.
- Mettre à jour le numéro de version selon l'ampleur du changement.
- Synchroniser les fichiers de modèles associés.
- Signaler les éléments laissés volontairement en attente.

Cette étape est importante, car une constitution mal synchronisée peut faire diverger les instructions données à l'IA et les templates utilisés pour les plans ou les spécifications.

## Ce que tu dois demander à l'IA

Pour obtenir une bonne modification, formule une demande **concrète** : quel principe changer, pourquoi, et avec quelle contrainte. Une bonne requête donne un objectif vérifiable, par exemple “ajoute une règle de testabilité pour les API” ou “renforce la compatibilité ascendante pour les migrations”.

- Indique le principe à modifier.
- Précise le motif métier ou technique.
- Mentionne si le changement est **majeur**, **mineur** ou **simple clarification**.
- Demande explicitement la mise en cohérence des templates.

Une fois la mise à jour terminée, Spec-Kit peut produire un **Sync Impact Report** pour lister les principes changés, les fichiers à ajuster, et les tâches restantes.

## 20. Valider les règles avant de spécifier

### Pourquoi valider avant de spécifier

Avant d'écrire une spécification, **les règles du projet doivent être relues, comprises et approuvées**. Dans Spec-Kit, la constitution sert de cadre non négociable : elle fixe ce que l'IA peut proposer, et ce qu'elle ne doit jamais contourner.

La logique est simple : **pas de spécification fiable sans règles stables**.

Cette validation précoce évite de produire des besoins qui contredisent l'architecture, la sécurité, la qualité ou les conventions déjà acceptées. Spec-Kit indique que la phase `/speckit.specify` ne fait qu'une *prise de conscience implicite*, alors que la vérification formelle arrive ensuite avec un contrôle explicite et bloquant dans `/speckit.plan`.

### Ce que signifie “valider les règles”

Valider les règles, c'est vérifier que la constitution est **cohérente, testable et applicable**. Une règle doit pouvoir être observée dans un artefact concret : code, plan, contrat, test ou documentation.

Les points à contrôler sont simples :

- la règle est **claire** et sans ambiguïté ;
- elle est **testable**, donc vérifiable par inspection ou analyse ;
- elle ne mélange pas principe et solution technique ;
- elle ne contredit pas une autre règle déjà écrite.

Par exemple, une bonne règle dit : *“toute fonctionnalité critique doit avoir des tests avant implémentation”*. Une mauvaise règle dit : *“le code doit être de bonne qualité”*, car cela reste trop vague pour être appliqué correctement.

### Quand faire cette validation dans Spec-Kit

La validation doit arriver **avant de rédiger une spécification détaillée**. La constitution est la première étape du flux Spec-Kit, puis les exigences sont construites en respectant ce socle.

Le contrôle des règles apparaît à plusieurs moments :

1. au moment où la constitution est lue ou créée ;
2. avant de lancer la planification ;
3. au moment de vérifier que la spécification n'introduit pas de violation.

Spec-Kit précise aussi qu'un plan peut être bloqué si une violation n'est pas justifiée, via un contrôle de conformité constitutionnelle. Cela signifie qu'une règle mal validée au départ peut empêcher toute la suite du travail.

## Comment vérifier concrètement les règles

Une méthode efficace consiste à relire chaque principe et à poser trois questions simples :

- **Est-ce formulé comme une règle stable ?**
- **Peut-on vérifier cette règle sans interprétation subjective ?**
- **Cette règle appartient-elle vraiment à la constitution, et non à la conception détaillée ?**

Exemple pratique : si la constitution impose une base de données approuvée, cette décision doit rester stable et vérifiable. En revanche, si elle décrit un schéma précis ou une requête SQL, elle devient trop technique pour une règle de niveau constitutionnel.

## Erreurs fréquentes à éviter

La première erreur est de laisser des formulations floues comme *“le système devrait être rapide”*. Une telle phrase n'est pas assez précise pour guider l'IA ou déclencher un contrôle sérieux.

La deuxième erreur est d'ajouter trop tôt des détails de conception, comme un framework précis, une structure de dossier ou une API. Les sources Spec-Kit rappellent que la constitution doit rester au niveau des principes, pas du mécanisme d'implémentation.

La troisième erreur est d'ignorer les désaccords entre règles. Si une règle de qualité contredit une contrainte technique, la validation doit les signaler avant d'écrire la spécification. C'est exactement le rôle du filtre constitutionnel : empêcher qu'une mauvaise base se transforme en mauvaise demande.

## 21. Décrire le besoin en langage naturel

### Décrire le besoin en langage naturel

Le point de départ consiste à formuler **ce que vous voulez construire** avec des mots simples, comme si vous l’expliquiez à un collègue non technique. Avec Spec-Kit, la commande `/specify` attend une description en langage naturel, centrée sur le **quoi** et le **pourquoi**, pas sur la solution technique.

Exemple de bonne formulation : “L’utilisateur doit pouvoir créer un compte, se connecter et réinitialiser son mot de passe.”

Exemple à éviter : “Utiliser une API REST avec authentification JWT.”

### Écrire comme une demande métier

Une bonne description commence par le **besoin réel** de la personne qui utilisera le logiciel. Il faut préciser le problème à résoudre, le contexte d’usage et l’objectif attendu, afin que l’IA construise une spécification utile sans deviner l’intention.

Par exemple, au lieu d’écrire “faire une page de profil”, préférez : “Permettre à un client de modifier son adresse et son numéro de téléphone depuis son espace personnel.” Cette formulation aide à identifier les fonctionnalités attendues sans entrer dans les détails d’implémentation.

### Ce qu’il faut inclure

Une description efficace contient généralement :

- **L’utilisateur cible** : qui fait l’action.
- **Le besoin** : ce qu’il veut accomplir.
- **Le résultat attendu** : ce qui doit être vrai après l’action.
- **Le contexte** : quand et pourquoi la fonctionnalité est utilisée.

Cette structure rend la demande plus exploitable par Spec-Kit, car la spécification générée pourra ensuite être enrichie avec des scénarios, des exigences et des critères d’acceptation.

### Ce qu’il faut éviter

À ce stade, il ne faut pas mentionner la **stack technique**, les bases de données, les frameworks ou les mécanismes d’implémentation. La documentation Spec-Kit insiste sur le fait de ne pas mélanger le besoin métier avec la solution technique, afin de garder une spécification claire et non biaisée.

Évitez aussi les formulations floues comme “améliorer l’expérience utilisateur” sans préciser ce que cela signifie concrètement. Une bonne pratique consiste à remplacer les mots vagues par des actions observables et des résultats vérifiables.

## **Exemple de formulation utile**

Une demande bien rédigée peut ressembler à ceci :

- “En tant qu’utilisateur, je veux recevoir une confirmation après l’envoi d’un formulaire pour savoir que ma demande a bien été prise en compte.”
- “Un administrateur doit pouvoir désactiver un compte afin de bloquer l’accès d’un utilisateur inactif.”
- “Le client doit pouvoir voir l’état de sa commande pour suivre son traitement.”

Ces phrases sont courtes, concrètes et compréhensibles. Elles donnent à l’IA une base solide pour rédiger une spécification fonctionnelle cohérente, tout en laissant la place aux étapes suivantes de clarification et d’organisation.

## 22. Structurer scénarios utilisateurs et exigences

### Scénarios utilisateurs

Un **scénario utilisateur** décrit, de façon concrète, ce qu’une personne veut accomplir avec la fonctionnalité. Dans Spec Kit, la spécification générée est structurée autour de *user scenarios*, d’exigences et de tests, afin de relier le besoin métier au comportement attendu du produit.

Un bon scénario répond à trois questions simples :

- **Qui** agit ?
- **Que** veut-il faire ?
- **Dans quel contexte** cela se produit-il ?

Par exemple : « *En tant que client, je veux réserver un créneau afin de confirmer mon rendez-vous en moins de deux minutes.* » Cette phrase est utile parce qu’elle décrit un objectif utilisateur, pas une solution technique.

### Exigences fonctionnelles

Une **exigence fonctionnelle** précise ce que le système doit faire pour satisfaire le scénario. Spec Kit génère des fichiers de spécification qui incluent ces exigences, souvent avec des scénarios de test et des critères d’acceptation.

Pour être exploitable, chaque exigence doit être :

- **claire** : une seule idée par phrase ;
- **vérifiable** : on peut dire si elle est respectée ou non ;
- **complète** : elle précise les règles, les cas limites et les exceptions ;
- **sans ambiguïté** : éviter les mots flous comme *rapide* ou *simple* sans définition.

Exemple : « *Le système doit afficher un message d’erreur si le créneau choisi est déjà réservé.* » Cette exigence est testable et directement reliée à une expérience utilisateur.

### Relier scénario et exigence

La structure la plus efficace consiste à partir d’un scénario, puis à le décliner en exigences. Spec Kit favorise cette logique en produisant une spécification structurée en user stories, critères d’acceptation et exigences fonctionnelles, ce qui aide l’IA à conserver le fil entre intention et implémentation.

Une méthode simple :

1. Décrire le besoin en langage utilisateur.
2. Découper le besoin en étapes observables.
3. Transformer chaque étape en exigence vérifiable.
4. Ajouter les erreurs, limites et états particuliers.

Exemple :

- Scénario : « *Je veux modifier mon mot de passe.* »
- Exigences : l'utilisateur doit confirmer son ancien mot de passe ; le nouveau mot de passe doit respecter les règles de sécurité ; un message de succès doit apparaître après validation.

## Cas limites et critères d'acceptation

Les **cas limites** sont les situations moins fréquentes mais importantes, comme une erreur réseau, un champ vide ou une autorisation refusée. Les documents Spec Kit recommandent de les capturer explicitement dans la spécification, avec les critères d'acceptation associés.

Un **critère d'acceptation** indique quand l'exigence est considérée comme remplie. Il doit rester mesurable :

- « *Le formulaire refuse l'envoi si l'e-mail est invalide.* »
- « *Le message de confirmation s'affiche après une réservation réussie.* »

Cette précision réduit les retours en arrière, car l'IA dispose d'un cadre explicite pour générer une solution conforme aux attentes.

## Écriture pratique dans Spec Kit

Dans Spec Kit, la commande de spécification produit un document qui doit rester centré sur le **quoi** et le **pourquoi**, pas sur la pile technique. C'est ici que vous formalisez les scénarios, les exigences et les critères de validation avant toute décision d'implémentation.

Pour écrire efficacement :

- utilisez des phrases courtes ;
- gardez un vocabulaire métier ;
- évitez les détails d'architecture ;
- relisez chaque exigence en demandant : « *Puis-je la tester ?* »

Exemple de formulation utile : « *L'utilisateur authentifié peut consulter son historique de commandes, filtré par date, et le système affiche un état vide si aucune commande n'existe.* » Cette phrase combine scénario, exigence et comportement attendu dans une forme directement exploitable par Spec Kit.

## 23. Lever les besoins de clarification

### Détecter ce qui manque

Dans Spec-Kit, **lever les besoins de clarification** signifie repérer les zones floues d’une spécification avant d’aller plus loin. La commande `/speckit.clarify` lance un échange structuré pour résoudre les ambiguïtés et les exigences insuffisamment détaillées, en posant jusqu’à cinq questions ciblées sur le contenu du `spec.md`.

L’idée centrale est simple : *si une décision n’est pas écrite, l’IA risque de la deviner.*

Exemples typiques de points à clarifier : le stockage des données, les permissions utilisateur, les comportements en erreur, ou les limites d’usage. Des guides Spec-Kit indiquent aussi que les marqueurs `need clarification` signalent précisément ces zones à compléter.

### Poser les bonnes questions

Une bonne clarification ne demande pas “tout expliquer”, mais *ce qui bloque une décision testable*. Spec-Kit analyse la spec et interroge les zones ambiguës, les cas limites et les contradictions possibles, puis intègre les réponses dans la section dédiée aux clarifications.

Les questions utiles portent souvent sur :

- les **règles métier** à confirmer ;
- les **cas d’erreur** à prévoir ;
- les **contraintes** de sécurité, de données ou de périmètre ;
- les **comportements attendus** quand plusieurs options sont possibles.

Cette approche évite les suppositions. Par exemple, “authentification” sans précision laisse ouverte la méthode ; “partage de photo” sans préciser local ou cloud crée une ambiguïté concrète à résoudre.

### Répondre sans inventer

Répondre à une clarification ne consiste pas à “choisir vite”, mais à écrire une décision claire, justifiable et exploitable. Spec-Kit recommande de consigner chaque réponse dans la spécification, afin que le document devienne la source de vérité du projet.

Une réponse efficace contient :

- la **décision retenue** ;
- la **raison** de ce choix ;
- l’**impact** sur le comportement du produit ;



- un point *ouvert* si une vérification reste nécessaire.

Par exemple : “Les fichiers sont stockés localement, car le produit cible un usage hors ligne.” Cette phrase réduit l’ambiguïté et donne à l’IA une base concrète pour la suite.

## Vérifier que la spec devient actionnable

Une spécification clarifiée doit permettre de passer à l’étape suivante sans interprétation libre. Selon la documentation Spec-Kit, `/speckit.clarify` s’utilise après la création de la spec et avant la planification, afin de stabiliser les exigences avant tout découpage technique.

Pour vérifier qu’une clarification est suffisante, demandez-vous :

- la règle peut-elle être **testée** ?
- le comportement est-il **observable** ?
- y a-t-il encore une **alternative implicite** ?
- une autre personne aurait-elle la même lecture du texte ?

Si la réponse est non, la clarification doit être réécrite. Cela transforme une idée vague en exigence exploitable, exactement ce que recherche une spec bien préparée.

## Pratiquer avec méthode

Dans un flux Spec-Kit, le bon réflexe est de traiter chaque doute comme une question à résoudre, pas comme un détail à ignorer. Les sources de formation et la documentation convergent : le but est de réduire les hypothèses et d’augmenter la précision avant l’exécution.

Un mini-processus simple fonctionne bien :

- relire la spec et repérer les formulations floues ;
- lancer `/speckit.clarify` ;
- répondre question par question ;
- réviser la spec jusqu’à suppression des zones incertaines.

Cette discipline améliore la qualité des spécifications et limite les écarts entre ce qui est demandé et ce qui sera construit.

## 24. Valider une spécification prête à planifier

### Valider la spécification avant de planifier

La **validation** consiste à vérifier qu'une spécification est assez claire, complète et cohérente pour passer à la phase **plan** sans deviner. Dans Spec Kit, l'objectif est de s'assurer que le futur plan technique pourra s'appuyer sur un document stable, lié à la spécification et aux principes du projet .

Une bonne spécification ne doit pas seulement décrire l'idée : elle doit permettre à un agent ou à un humain de préparer un plan sans ambiguïté.

### Ce qu'il faut contrôler en priorité

Une spécification prête à planifier doit répondre à quatre questions simples : **quoi, pour qui, dans quel cadre, et comment savoir que c'est réussi**. Les sources sur Spec Kit insistent sur la présence d'objectifs, de portée, de contraintes et de critères d'acceptation avant de passer au plan.

- Les besoins utilisateur sont formulés clairement.
- Le périmètre est explicite, avec ce qui est inclus et exclu.
- Les contraintes fonctionnelles ou de conformité sont visibles.
- Les critères d'acceptation sont testables et concrets.

Un exemple utile : au lieu d'écrire « l'application doit être rapide », il faut préciser un critère observable, comme « l'écran principal doit s'afficher en moins de 2 secondes dans le cas nominal ». Cette forme aide l'étape /plan à produire une architecture adaptée, sans supposer ce que signifie “rapide”.

### Repérer les zones floues

La validation sert aussi à détecter les trous de spécification. Spec Kit met l'accent sur la cohérence entre les artefacts, et plusieurs guides recommandent de rechercher les contradictions, doublons et omissions avant l'implémentation.

- Vérifier qu'aucune fonctionnalité n'est décrite deux fois avec des termes différents.
- Vérifier qu'aucune dépendance critique n'est absente, comme un service externe ou une contrainte de sécurité.
- Vérifier que les cas limites sont mentionnés, par exemple erreur réseau, données absentes ou droits insuffisants.
- Vérifier que les critères d'acceptation couvrent chaque besoin principal.

Cette étape évite qu'un plan technique soit construit sur une base fragile. Si une exigence n'est pas validée, le plan risque de combler les vides par des hypothèses, ce qui contredit l'approche pilotée par la spécification.

## Utiliser une liste de validation simple

Une spécification prête à planifier peut être relue avec une checklist courte et pratique . Les sources sur Spec Kit mentionnent justement des critères de revue et d'acceptation pour s'assurer que l'entrée dans la phase suivante est saine.

1. Les objectifs métier sont compréhensibles.
2. Les utilisateurs cibles sont identifiés.
3. Les exigences sont mesurables.
4. Les contraintes sont explicites.
5. Les critères d'acceptation existent pour chaque exigence importante.
6. Les ambiguïtés restantes sont marquées comme questions ouvertes.

Par exemple, si la spécification dit seulement « permettre l'authentification », elle n'est pas encore prête. Il faut savoir s'il s'agit d'un mot de passe, d'un SSO, d'une connexion par lien magique ou d'un autre mécanisme, car le plan technique dépend directement de ce choix.

## Décider si la spécification peut passer au plan

Quand la validation est réussie, la spécification devient une base de travail fiable pour /**plan**, qui transforme le “quoi” en direction technique concrète. À ce stade, l'équipe peut choisir l'architecture, les contraintes et les dépendances sans corriger d'abord des ambiguïtés de fond.

Le bon test est simple : si une autre personne peut lire la spécification et produire le même besoin sans interprétation majeure, alors elle est *prête à planifier*. Si ce n'est pas le cas, il faut encore clarifier avant d'avancer .

## 25. Choisir technologies sans deviner tôt

### Choisir les technologies au bon moment

Dans Spec Kit, le choix des technologies se fait **au moment du plan technique**, pas au moment de la spécification. Cette séparation évite de mélanger le besoin métier avec la solution technique, ce qui rend le projet plus clair et plus stable.

L'idée simple : d'abord dire **ce qu'il faut construire**, puis décider **comment le construire**.

### Pourquoi ne pas deviner trop tôt

Choisir un framework, une base de données ou une architecture trop tôt pousse souvent à corriger ensuite des décisions prises sans contexte complet. Spec Kit recommande donc de garder la phase `/specify` sans détails techniques, puis d'ouvrir la phase `/plan` quand les besoins sont validés.

Cette méthode est utile parce qu'elle aide l'IA à travailler sur des faits précis plutôt que sur des suppositions. Le plan devient alors un document de référence pour la suite du projet.

### Ce qu'on décide dans `/plan`

La phase `/plan` sert à fixer la **stack technique**, c'est-à-dire l'ensemble des technologies utilisées pour réaliser le logiciel : langage, framework, base de données, API, outils de test et contraintes d'intégration.

On peut y préciser, par exemple :

- **Node.js** pour le backend
- **Python** pour un service de traitement
- **SQLite** pour un prototype
- **PostgreSQL** pour une base plus robuste
- **Manifest V3** pour une extension navigateur

Le plan généré par Spec Kit peut inclure `plan.md`, qui décrit l'architecture et les décisions techniques, ainsi que d'autres artefacts comme `research.md` ou `data-model.md` selon le projet.

## Comment choisir sans improviser

Le bon choix technique dépend de contraintes concrètes, pas de préférences abstraites. Spec Kit recommande de partir de critères comme la compatibilité, les performances, la maintenance et les standards déjà présents dans l’environnement cible.

Quelques critères simples :

- **Compatibilité** avec l’existant
- **Simplicité** pour l’équipe
- **Performance** attendue
- **Sécurité** requise
- **Évolutivité** à moyen terme

Par exemple, une petite application interne peut justifier SQLite, alors qu’un produit multi-utilisateurs avec charge croissante s’oriente plus souvent vers PostgreSQL. Cette décision doit être écrite dans le plan, pas laissée à l’IA au moment du code.

## Un exemple de formulation utile

Une bonne entrée de /plan ressemble à une demande structurée, avec choix techniques et contraintes explicites. Cela permet à l’IA de produire un plan d’implémentation cohérent au lieu d’inventer une architecture au fil de l’eau.

Exemple :

- application web
- backend en Node.js
- base PostgreSQL
- authentification par JWT
- contraintes de conformité et de performance
- interface compatible avec l’existant métier

Cette approche transforme le choix technologique en décision documentée, vérifiable et réu-

tilisable. Elle réduit les ambiguïtés et prépare un plan technique exploitable par la suite.

## 26. Générer plan recherche et modèle de données

### Préparer la recherche technique

Le **plan de recherche** sert à transformer une spécification fonctionnelle en questions techniques concrètes. Dans Spec Kit, `/speckit.plan` produit un document de recherche qui analyse les standards, compare les options et justifie les choix avant l'implémentation.

Pensez-y comme à une *enquête courte mais structurée* : on ne code pas encore, on vérifie ce qui est viable

Pour le rédiger, partez de la spec et listez les zones d'incertitude :

- contraintes de performance,
- sécurité,
- persistance des données,
- intégration avec des services externes,
- format des échanges entre composants.

Un bon exemple est de demander : *comment l'application stocke-t-elle une image, un document, ou un état utilisateur sans casser le flux métier ?* Spec Kit attend justement ce type d'analyse dans `research.md`.

### Structurer `research.md`

Le fichier `research.md` explique *pourquoi* une solution a été choisie. Selon la documentation Spec Kit, `/speckit.plan` génère un plan complet avec des documents de soutien sur les modèles de données, les contrats d'API et les scénarios de test.

Ce document doit contenir :

- les hypothèses de départ,
- les standards ou conventions retenus,
- les alternatives écartées,
- les risques connus,
- les décisions avec leur justification.

Par exemple, si deux options existent pour gérer le stockage, le plan de recherche compare leur simplicité, leur maintenance et leur compatibilité avec la spec. Cette logique évite les choix "par habitude" et rend le projet traçable .

## Construire le modèle de données

Le **modèle de données** décrit la structure des informations manipulées par l'application. Spec Kit précise qu'un fichier comme `data-model.md` doit documenter la structure des données, les relations entre entités et les règles de validation.

Dans la pratique, on y définit :

- les entités principales,
- leurs attributs,
- leurs relations,
- les champs obligatoires,
- les contraintes de format.

Exemple simple : pour une fiche produit, on peut distinguer **Produit**, **Image**, **Catégorie** et **Stock**. Une relation **Produit** → **Image** indique qu'un produit peut avoir plusieurs images, tandis qu'une validation peut imposer qu'un nom ne soit jamais vide. Cela rend la base logique avant même d'écrire le code .

## Relier recherche et données à la spec

Le point clé de Spec Kit est la cohérence entre la spec, la recherche et le modèle de données. Le flux décrit par GitHub suit l'enchaînement **Spec** → **Plan** → **Tasks** → **Implement**, et le plan doit traduire les exigences métier en architecture technique.

Pour rester aligné :

- chaque besoin fonctionnel doit avoir une réponse technique,
- chaque donnée stockée doit servir un scénario utilisateur,
- chaque règle de validation doit correspondre à une exigence de la spec.

Si la spec demande un historique des actions, le modèle de données doit prévoir une entité d'audit. Si la spec impose des pièces jointes, la recherche doit vérifier les stratégies de stockage adaptées. Cette correspondance directe réduit les oublis et limite les reprises tardives .

## Écrire un plan exploitable

Un bon plan technique ne reste pas théorique. Il doit être assez précis pour guider la suite, tout en laissant la place aux décisions détaillées lors de l'implémentation. Spec Kit génère aussi des éléments comme `quickstart.md`, ce qui montre que le plan sert de base opérationnelle et non de simple note d'intention.

Pour tester la qualité du plan, vérifiez :



- une décision est-elle justifiée par une contrainte réelle ?
- le modèle de données couvre-t-il tous les cas d'usage ?
- une personne tierce peut-elle comprendre le système sans deviner ?

Si la réponse est oui, votre recherche et votre modèle de données sont prêts à soutenir un développement piloté par spécification, clair et robuste

## 27. Définir contrats API et accès fichiers

### Définir les contrats

Un **contrat API** décrit précisément ce qu'un service accepte et renvoie : routes, méthodes HTTP, paramètres, schémas de données et erreurs. Dans Spec-Kit, ce niveau de détail aide l'IA à produire un plan technique cohérent, sans inventer des comportements non spécifiés.

Pour un projet logiciel, commencez par fixer les opérations essentielles : lire, créer, modifier ou supprimer. Par exemple, `GET /contracts/{id}/analysis` peut renvoyer une analyse structurée, tandis que `POST /contracts/ingest` peut importer un document et déclencher son traitement.

Pensez au contrat API comme à un *mode d'emploi partagé* entre le produit, le backend et les futurs intégrateurs

### Spécifier les entrées et sorties

Chaque endpoint doit indiquer ses **entrées** : paramètres de chemin, corps JSON, headers, authentification, pagination ou filtres. Il doit aussi préciser ses **sorties** : codes de statut, structure de réponse, champs obligatoires et champs optionnels. Cette précision réduit les ambiguïtés lors de la mise en œuvre.

Exemple simple :

```
{
  "contractId": "ctr_123",
  "analysis": {
    "risks": [],
    "obligations": [],
    "gaps": []
  }
}
```

Ajoutez toujours les erreurs attendues, par exemple 401 pour une authentification absente, 403 pour un accès interdit, ou 422 pour des données invalides. Les droits d'accès doivent apparaître dans la spécification, pas seulement dans le code.

### Encadrer l'accès aux fichiers

L'accès aux fichiers doit être traité comme une partie du contrat, pas comme un détail d'implémentation. Spécifiez quels formats sont acceptés, comment les fichiers sont transmis, et quand ils deviennent disponibles après traitement. OpenAI indique, par exemple, que

`file_search` s'appuie sur un **Vector Store** et que les fichiers doivent être attendus jusqu'à la fin du traitement.

Définissez aussi les règles de sécurité : taille maximale, types MIME autorisés, droits par rôle et durée de conservation. Pour des contenus textuels, les formats courants incluent `text/plain`, `application/json`, `text/markdown` ou `application/pdf`.

## Décrire les flux de traitement

Quand un fichier est chargé, le contrat doit dire ce qui se passe ensuite : stockage, indexation, analyse, puis récupération. Si le système utilise un outil de recherche documentaire, la spécification doit mentionner l'objet de stockage, l'attente de fin de traitement et la manière d'associer le contenu à l'assistant ou au service.

Une bonne pratique consiste à écrire un scénario complet :

1. L'utilisateur envoie un PDF.
2. Le backend valide le format.
3. Le fichier est stocké.
4. Le système lance l'analyse.
5. L'API renvoie un identifiant de suivi.

## Rendre le contrat exploitable par le plan

Le plan technique doit reprendre chaque contrat sans le réinterpréter. Indiquez les choix d'architecture, les dépendances minimales et les limites de périmètre, comme le recommande Spec-Kit avec son flux **Spec** → **Plan** → **Tasks**.

Pour rester clair, reliez chaque endpoint à une responsabilité unique : ingestion, consultation, ou gestion des fichiers. Cette séparation facilite les tests, la maintenance et la génération des tâches techniques, tout en gardant le périmètre lisible pour l'IA.

## 28. Analyser cohérence spec plan et qualité

### Vérifier l'alignement entre spécification et plan

La cohérence commence par un contrôle simple : chaque exigence de la spécification doit retrouver une réponse explicite dans le plan technique, et chaque action du plan doit être justifiée par une exigence réelle. Un plan qualité de projet doit rester cohérent avec son périmètre, ses données d'entrée et les besoins des utilisateurs prévus.

Dans un contexte Spec-Kit IA, cela veut dire que le document de plan ne doit pas inventer de travail inutile ni oublier des contraintes déjà écrites. Un plan qualité formalise justement les dispositions particulières pour atteindre les exigences contractuelles de qualité.

- Relier chaque exigence à une décision du plan
- Repérer les exigences sans traitement
- Repérer les tâches sans source dans la spec

Une bonne vérification de cohérence répond à une question très concrète : *est-ce que le plan fait exactement ce que la spécification demande, ni plus ni moins ?*

### Contrôler la complétude des exigences

La complétude consiste à vérifier que rien d'important n'a disparu entre la spec et le plan. Les guides qualité indiquent qu'un plan doit couvrir les objectifs qualité, les responsabilités, les ressources, les mesures, les vérifications et le traitement des non-conformités.

Pour l'IA appliquée au logiciel, cette vérification est essentielle, car un modèle peut produire un plan plausible mais incomplet. Il faut donc comparer la spec avec une grille de couverture, par exemple :

1. objectifs fonctionnels
2. exigences non fonctionnelles
3. contraintes de sécurité
4. critères d'acceptation
5. modalités de test et de revue

Un exemple simple : si la spec impose une traçabilité des actions utilisateur, le plan doit prévoir le stockage des logs, leur conservation et leur contrôle qualité. Sans cela, la cohérence est cassée.

## Tester la compatibilité des choix techniques

La compatibilité vérifie que les solutions proposées peuvent réellement satisfaire les exigences. ISO 10005 insiste sur la cohérence et la compatibilité du contenu du plan avec son champ d'application et ses entrées. Le plan qualité doit aussi rester compatible avec les processus de l'organisation et ses standards internes.

Dans un plan généré par IA, un bon réflexe consiste à repérer les contradictions internes :

- une exigence de haute disponibilité avec une architecture mono-serveur
- un besoin de chiffrement avec des fichiers stockés en clair
- des tests définis sans environnement de test
- des responsabilités floues alors que le plan doit préciser le *qui fait quoi*

Cette lecture critique évite les plans “propres sur le papier” mais impossibles à exécuter.

## Mesurer la qualité du plan lui-même

La qualité ne concerne pas seulement le produit final : elle concerne aussi le plan. Les sources qualité recommandent une revue, une acceptation, une application et une révision formelles du plan. Cela permet de valider que les étapes sont cohérentes et aptes à répondre aux exigences.

Concrètement, on évalue :

- la clarté des responsabilités
- la mesurabilité des critères
- la présence d'indicateurs et de seuils
- la traçabilité entre exigences, actions et contrôles
- la gestion des écarts et des corrections

Exemple utile : si le plan dit “qualité élevée” sans définir de métrique, il est faible. Si, au contraire, il précise un taux d'échec maximal de test, un délai de correction et un responsable de validation, il devient exploitable.

## Mettre en place une revue critique avec l'IA

L'IA est utile pour détecter les incohérences, mais elle ne remplace pas la revue humaine. Un plan de qualité doit rester documenté, vérifiable et révisable, avec des éléments explicites sur les mesures, les données, les responsabilités et les contrôles.

Une méthode simple consiste à demander à l'IA de produire une matrice de cohérence, puis à valider manuellement trois points :

- chaque exigence a une réponse
- chaque tâche a un objectif qualité
- chaque contrôle a un responsable et une preuve attendue

Cette double lecture est la meilleure défense contre les oublis, les ambiguïtés et les contradictions. Elle transforme un plan “généré” en plan réellement fiable

## 29. Transformer le plan en tâches actionnables

### Décomposer un plan en tâches exécutables

Transformer un plan en tâches actionnables consiste à passer d’une vision d’architecture à une *liste d’actions concrètes*, assez petites pour être réalisées, testées et relues sans ambiguïté. Spec Kit formalise ce passage avec la commande `/speckit.tasks`, qui génère une liste de tâches exploitables à partir du plan validé.

Une bonne tâche doit répondre à une seule question : *quelle action précise faut-il faire maintenant ?*

### Ce qu’une tâche doit contenir

Une tâche actionnable décrit **quoi faire**, **sur quoi agir** et **comment vérifier le résultat**. Dans les guides autour de Spec Kit, la décomposition attendue inclut typiquement une description, des dépendances, des livrables et des conditions d’acceptation. Cette structure évite les tâches vagues comme “mettre en place l’API” et les remplace par des unités vérifiables .

- **Description** : l’action à réaliser
- **Dépendances** : ce qui doit être terminé avant
- **Livrable** : ce qui doit exister à la fin
- **Acceptation** : le critère qui prouve que la tâche est terminée

### Découper finement sans perdre le fil

Le bon niveau de granularité est celui qui permet à une tâche d’être terminée rapidement et validée clairement. Spec Kit indique que la liste doit être *actionnable* et ordonnée à partir du plan d’implémentation. En pratique, une grosse étape devient plusieurs sous-actions indépendantes, par exemple : créer le schéma de données, écrire les tests, puis connecter le service.

- Éviter les tâches trop larges : elles cachent plusieurs décisions
- Éviter les tâches trop petites : elles ralentissent la lecture
- Garder un ordre logique : préparation, construction, vérification
- Faire apparaître les dépendances explicites

### Exemple concret de transformation

Un plan technique peut dire : “ajouter un stockage de fichiers et exposer un endpoint de téléchargement”. La transformation en tâches devient plus précise :

1. Définir le contrat de stockage des fichiers
2. Implémenter la couche d'accès au stockage
3. Créer l'endpoint de téléchargement
4. Ajouter les tests unitaires et d'intégration
5. Vérifier les erreurs d'accès et les cas limites

Cette logique correspond à l'objectif de `/speckit.tasks` : produire une suite de tâches ordonnées à partir du plan, afin que l'implémentation puisse démarrer sans re-clarification continue.

## Bonnes pratiques pour des tâches utiles

Une tâche bien écrite doit être formulée comme un *verbe d'action* suivi d'un résultat observable. Le framework de prompt engineering utilisé dans plusieurs ressources insiste aussi sur la précision de l'action et du format attendu, ce qui améliore la qualité de sortie d'un modèle. Ici, la clarté est plus importante que la sophistication .

- Employer des verbes concrets : créer, relier, valider, corriger
- Nommer les objets techniques : endpoint, schéma, test, composant
- Ajouter des critères de succès mesurables
- Prévoir les blocages potentiels avant de coder

## Vérifier que la liste est exploitable

Une bonne liste de tâches permet à une équipe ou à un agent IA d'avancer sans interprétation excessive. Si une tâche ne peut pas être testée, si elle dépend d'hypothèses implicites ou si elle regroupe plusieurs livrables, elle doit être reformulée. C'est précisément ce qui rend le découpage actionnable : chaque ligne devient une instruction de travail claire et vérifiable.



## 30. Ordonner dépendances et jalons testables

### Dépendances d'abord, exécution ensuite

Dans un travail découpé en tâches, une **dépendance** signifie qu'une tâche ne peut pas commencer tant qu'une autre n'est pas terminée. Cette règle sert à éviter les blocages et à ordonner le flux de travail de façon logique, comme une chaîne d'assemblage .

Pour une spécification IA, l'ordre doit suivre les **entrées** et **sorties** de chaque tâche : on commence par ce qui produit un résultat réutilisable, puis on enchaîne avec ce qui consomme ce résultat.

### Identifier les types de liens

On distingue surtout trois relations utiles :

- **Bloque** : une tâche empêche une autre de démarrer.
- **Bloqué par** : une tâche attend une autre tâche.
- **Dépendance mutuelle à éviter** : deux tâches se renvoient la balle, ce qui crée un cycle impossible à exécuter.

Une bonne méthode consiste à lire chaque tâche comme une phrase simple : “*Que faut-il avoir fini avant de commencer ?*” Si la réponse est claire, la dépendance est correcte. Si plusieurs réponses se contredisent, la tâche est probablement trop large.

### Placer les jalons testables

Un **jalon** est un point de contrôle sans durée propre ; il marque la fin d'une phase ou la validation d'un livrable . Dans un plan de tâches, un jalon devient utile seulement s'il peut être **vérifié** de manière concrète, par exemple avec une démonstration, une revue, ou un test d'acceptation.

Exemples de jalons testables :

- *API prête* : une requête de test renvoie une réponse conforme au contrat.
- *Accès fichiers validé* : lecture et écriture fonctionnent sur un répertoire de test.
- *Intégration terminée* : l'enchaînement de deux tâches produit le résultat attendu.

Un jalon n'est donc pas une vague intention ; c'est un critère observable, souvent lié à un livrable ou à une validation formelle.

## Ordonner par couches de livraison

Pour éviter un plan confus, classe les tâches par couches :

1. **Fondations** : ce qui prépare l'environnement ou les interfaces.
2. **Capacités de base** : ce qui permet de faire le premier flux complet.
3. **Validation** : ce qui prouve que le flux fonctionne.
4. **Durcissement** : ce qui réduit les erreurs et stabilise le résultat.

Cette logique suit le principe de planification qui consiste à séquencer les activités selon leurs liens directs, puis à insérer les jalons pour matérialiser les points de passage clés.

## Vérifier que le plan reste exécutable

Un plan bien ordonné doit rester lisible par un humain et par un outil. Dans les outils de gestion, les dépendances sont souvent affichées par des lignes ou des liens entre tâches et jalons, ce qui aide à contrôler l'ordre d'exécution.

Pour tester la qualité de l'ordonnancement, pose-toi ces questions :

- Une tâche peut-elle démarrer sans information manquante ?
- Chaque jalon peut-il être validé par un test clair ?
- Existe-t-il une tâche sans dépendance inutile ?
- Le plan contient-il un cycle caché ?

Si la réponse est non à la dernière question, le séquençage est sain. Si un jalon n'est pas testable, il doit être reformulé jusqu'à devenir mesurable, vérifiable et utile au pilotage du travail.

## 31. Estimer complexité et risques techniques

### Évaluer la charge technique

La **complexité technique** mesure l'effort réel nécessaire pour réaliser une tâche sans casser l'existant. Dans Spec-Kit, le plan technique doit déjà fixer la stack, l'architecture et les contraintes, puis la phase de tâches découpe ce travail en unités petites et testables.

Un bon repère consiste à distinguer ce qui est *simple à coder* de ce qui est *difficile à intégrer*. Par exemple, ajouter un champ de formulaire est souvent faible complexité, tandis que connecter une API, gérer l'authentification et les erreurs réseau augmente nettement la charge.

### Identifier les facteurs de complexité

Pour estimer proprement, examinez chaque tâche selon plusieurs facteurs concrets :

- **Dépendances** : la tâche dépend-elle d'un autre composant déjà prêt ?
- **Incertitude** : des zones floues restent-elles dans la spec ou le plan ?
- **Intégration** : faut-il toucher plusieurs services, bases de données ou interfaces ?
- **Contraintes de qualité** : tests, sécurité, performance, conformité.
- **Compétence requise** : la tâche demande-t-elle une expertise rare ou une bibliothèque inconnue ?

Une tâche est plus coûteuse dès qu'elle combine plusieurs de ces facteurs. L'idée n'est pas de deviner au hasard, mais de classer le travail selon sa *densité de risque* et son *niveau d'interaction* avec le reste du système.

### Catégoriser les risques techniques

Les risques techniques se classent souvent en quatre familles : **données**, **modèles**, **opérationnels** et **sécurité**. Cette grille est utile en IA, car les systèmes peuvent échouer à cause de données de mauvaise qualité, de comportements imprévus du modèle, ou de vulnérabilités exposées au déploiement.

En pratique, cela donne des exemples très concrets :

- données incomplètes ou biaisées ;
- sorties non fiables ou instables ;
- erreurs d'intégration avec l'existant ;
- failles de sécurité, attaques adverses ou extraction du modèle ;
- non-conformité réglementaire.

L'ANSSI recommande de mener l'analyse de risque *le plus tôt possible* et de prévoir des tests de robustesse et de sécurité avant déploiement. Cela veut dire qu'une tâche n'est pas seulement "difficile" : elle peut aussi être *dangereuse* si elle touche à la sûreté ou à la qualité du service.

## Donner un niveau simple à chaque tâche

Une méthode pratique consiste à noter chaque tâche sur trois niveaux : **faible**, **moyen**, **élevé**. Une tâche faible a un périmètre clair, peu de dépendances et des tests simples. Une tâche élevée implique plusieurs systèmes, des risques de régression et des validations spécialisées.

Exemple :

- **Faible** : corriger un libellé ou un message d'erreur.
- **Moyen** : exposer une nouvelle route API avec tests.
- **Élevé** : ajouter un composant IA qui influence une décision métier, avec surveillance, robustesse et sécurité.

Si une tâche est élevée, il faut souvent la découper encore avant de l'assigner.

## Transformer le risque en jalons testables

Le bon réflexe est de convertir chaque risque en **jalon testable**. Par exemple, un risque d'incohérence peut devenir un test d'acceptance ; un risque de sécurité peut devenir un audit ciblé ; un risque de performance peut devenir un seuil mesurable.

Cela permet de garder le travail pilotable : chaque sous-tâche doit pouvoir être vérifiée sans attendre la fin du projet. Dans Spec-Kit, cette logique protège la suite du workflow, car la liste de tâches ordonnée doit rester réaliste, testable et compatible avec les contraintes du plan.

## 32. Réviser la liste des tâches avec l'IA

### Réviser la liste des tâches avec l'IA

La **révision** de la liste des tâches consiste à vérifier que chaque tâche est claire, testable et reliée à une exigence précise. Dans Spec Kit, l'étape `/speckit.tasks` génère une liste exploitable à partir du plan, puis l'IA peut aider à la corriger avant l'implémentation.

L'objectif n'est pas d'ajouter plus de tâches, mais de rendre chaque tâche **actionnable** et **sans ambiguïté**.

### Ce que l'IA doit vérifier

L'IA relit les artefacts précédents, comme `spec.md` et `plan.md`, puis contrôle si les tâches couvrent bien les besoins décrits.

- chaque tâche correspond à une seule action identifiable ;
- les dépendances sont dans le bon ordre ;
- les tâches sont assez petites pour être testées séparément ;
- aucune exigence importante n'est oubliée.

Une tâche trop vague, comme *“améliorer l'interface”*, doit être reformulée en quelque chose de précis, par exemple *“ajouter un formulaire de connexion avec validation des champs”*. Cela aide l'agent à travailler comme un exécutant méthodique, pas comme un improvisateur.

### Comment réviser efficacement

La bonne méthode consiste à demander à l'IA de comparer la liste des tâches avec les fichiers de conception déjà produits. Spec Kit prévoit justement que l'agent lise les artefacts du projet pour produire ou corriger la liste de travail.

1. relire les tâches une par une ;
2. repérer les doublons, oublis ou formulations floues ;
3. vérifier que les tâches suivent les dépendances techniques ;
4. reformuler les tâches pour qu'elles soient vérifiables par un test ou une validation manuelle.

Par exemple, si une tâche dépend d'un schéma de données non encore défini, elle doit être déplacée après la création de ce schéma. Cette logique évite les blocages pendant l'implémentation.

## Critères d’une bonne tâche

Une tâche bien révisée doit être **atomique**, c’est-à-dire assez petite pour être réalisée sans mélange de responsabilités. Elle doit aussi être **observables**, donc facile à confirmer par un résultat concret, comme un test vert ou un fichier généré.

- **atomique** : une seule responsabilité ;
- **vérifiable** : on peut dire si elle est terminée ;
- **ordonnée** : elle respecte les dépendances ;
- **traçable** : elle renvoie à une exigence du plan.

Dans Spec Kit, cette discipline facilite le passage vers `/speckit.implement`, qui exécute les tâches dans l’ordre défini.

## Exemples de révision avec l’IA

Une tâche mal rédigée :

- “Créer le *backend*”

Version révisée :

- “Créer l’API de gestion des utilisateurs avec validation d’entrée et tests unitaires”

Une tâche trop large :

- “Préparer l’authentification”

Version révisée :

- “Implémenter le point de terminaison de connexion et vérifier le retour d’erreur en cas de mot de passe invalide”

Ce travail de révision améliore la qualité du `tasks.md` et réduit les corrections tardives. Dans la logique Spec Kit, l’IA sert ici de relecteur structurant, capable de transformer un plan général en suite d’actions claires, petites et testables.

## 33. Exécuter les tâches étape par étape

### Exécuter la liste de tâches

L'étape **exécuter les tâches étape par étape** consiste à prendre la liste issue de `/speckit-tasks` et à la traiter **dans l'ordre**, sans sauter de bloc. Spec Kit décrit ensuite l'implémentation comme une phase où l'agent code avance sur les tâches **une par une**, ou en parallèle seulement quand cela est explicitement possible.

L'idée centrale est simple : **une tâche = un résultat vérifiable**. Si une tâche n'est pas terminée, on ne la considère pas comme acquise.

### Commencer petit et vérifier souvent

Une bonne exécution ne vise pas à “tout faire d'un coup”. GitHub indique que les tâches doivent être des morceaux **petits, révisables et actionnables** ; cela permet à l'agent de produire un changement précis, puis de le faire contrôler avant de passer au suivant.

Concrètement, tu peux suivre cette logique :

1. Choisir la première tâche non bloquée.
2. La confier à l'agent avec le contexte minimal utile.
3. Vérifier le résultat dans les fichiers sources.
4. Marquer la tâche comme faite seulement si le comportement attendu est visible.

### Garder l'agent aligné sur le projet

Pendant l'exécution, l'agent doit rester **ancré dans la spec, le plan et la liste de tâches**. Le workflow Spec Kit est justement conçu comme une chaîne séquentielle : *specify* → *plan* → *tasks* → *implement*, afin d'éviter qu'un agent invente une solution hors cadre.

Pour un développeur, cela veut dire :

- relire la tâche exacte avant chaque action ;
- vérifier que le code modifié correspond bien au besoin ;
- éviter d'ajouter des fonctionnalités non demandées ;
- revenir au plan si un écart apparaît.

### Exemple d'exécution progressive

Imagine une fonctionnalité avec trois tâches : créer un endpoint, valider l'entrée, puis écrire le test. L'agent commence par le endpoint, car c'est la base ; ensuite seulement il ajoute

la validation, puis les tests. Cette progression réduit les erreurs et rend chaque étape plus simple à contrôler.

Un bon prompt d'exécution ressemble à ceci :

Prends la tâche 1 seulement.

Modifie uniquement les fichiers nécessaires.

Explique brièvement ce qui a été changé.

N'avance pas à la tâche suivante sans validation.

## Contrôler, corriger, continuer

Spec Kit inclut une phase de **convergence** après l'implémentation, ce qui montre que l'exécution n'est pas seulement du codage : elle doit être suivie d'une vérification contre la spec, le plan et les tâches.

Pour rester efficace, applique ce cycle simple :

- exécuter ;
- observer le résultat ;
- corriger les écarts ;
- passer à la tâche suivante seulement quand le comportement est confirmé.

Cette méthode rend l'agent plus prévisible, et elle aide à conserver des changements **traçables** et **testables** à chaque étape.



## 34. Garder l’IA alignée sur fichiers sources

### Garder l’IA alignée sur les fichiers sources

L’idée centrale est simple : **l’agent doit modifier le projet en respectant les fichiers déjà présents**, pas réécrire le besoin de mémoire. Dans Spec Kit, l’alignement vient du fait que l’agent lit et met à jour les artefacts du projet — comme `spec.md`, `plan.md` et `tasks.md` — au lieu d’inventer une implémentation isolée.

Le bon réflexe n’est pas “générer du code vite”, mais “faire correspondre chaque action de l’agent à une source de vérité déjà écrite”.

### S’appuyer sur les sources de vérité

Spec Kit organise le travail autour d’un dossier `.specify` qui contient les modèles, scripts et principes du projet, tandis que les fichiers de spécification décrivent quoi construire et comment le faire par étapes. La documentation indique aussi que l’initialisation crée les bons fichiers et dossiers selon l’agent choisi, ce qui réduit les écarts de format entre l’intention et l’exécution.

Pour rester aligné, l’agent doit toujours consulter les fichiers existants avant d’agir. Concrètement, cela signifie :

- lire `spec.md` pour le besoin fonctionnel ;
- lire `plan.md` pour l’architecture et les choix techniques ;
- lire `tasks.md` pour l’ordre d’exécution et les dépendances ;
- ne modifier que ce qui est nécessaire dans les fichiers sources du projet.

### Faire travailler l’agent sur les bons fichiers

Spec Kit prévoit des intégrations pour des agents IA différents, avec des structures et commandes adaptées à chacun. La documentation précise que ces intégrations installent les bons fichiers de commande et la bonne arborescence pour l’agent ciblé. Cela évite qu’un agent écrive dans un dossier qui ne correspond pas au projet réel.

Un bon usage ressemble à ceci :

1. lancer l’initialisation du projet avec l’intégration correcte ;
2. générer les artefacts dans `.specify` ;
3. demander à l’agent d’exécuter les tâches en lisant ces fichiers ;
4. vérifier que chaque modification correspond à une tâche existante.

## Réduire les dérives pendant l'implémentation

L'alignement se perd surtout quand l'agent improvise. Spec Kit limite ce risque en séparant les rôles : les fichiers de spécification définissent la cible, et la phase d'implémentation exécute seulement ce qui a été décidé. Plusieurs guides indiquent que la commande d'implémentation doit d'abord vérifier la présence des artefacts attendus avant de construire la fonctionnalité.

Pour éviter les dérives, il faut aussi garder les fichiers cohérents entre eux. Si `tasks.md` mentionne une dépendance, l'agent ne doit pas la contourner ; s'il existe une règle dans `constitution.md`, elle doit rester applicable à chaque modification. Cette logique de continuité transforme l'agent en exécutant guidé, pas en auteur autonome.

## Contrôler la correspondance avec les sources

Garder l'IA alignée sur les fichiers sources, c'est enfin vérifier régulièrement que le résultat produit suit bien les documents du projet. La documentation et les retours d'expérience montrent que la séquence `spec` → `plan` → `tasks` → `implement` sert précisément à maintenir cette correspondance.

Un contrôle efficace repose sur trois vérifications :

- le code reflète bien les tâches écrites ;
- les tâches reflètent bien le plan ;
- le plan reflète bien la spécification.

Quand cette chaîne est respectée, l'agent travaille comme un traducteur fidèle entre intention et code, ce qui rend l'implémentation plus prévisible et plus facile à corriger.

## 35. Gérer le contexte et éviter les dérives

### Garder le contexte petit et utile

Le **contexte** est tout ce que l'agent lit avant d'agir : spécification, plan, tâches, règles du projet et fichiers sources. Avec Spec-Kit, l'objectif est de transformer ce contexte en artefacts structurés, afin d'éviter les prompts improvisés et les oublis coûteux.

Plus le contexte est clair, moins l'agent invente. Plus il est compact, moins il se disperse.

### Ce qu'il faut faire entrer dans l'agent

Commence par n'envoyer que l'essentiel : **la spec**, **le plan**, puis **la liste de tâches**. Spec Kit suit précisément cette chaîne "Spec → Plan → Tasks → Implement", et chaque étape produit un document Markdown qui nourrit la suivante.

- Décrire le **quoi** dans `/speckit.specify`, pas le comment.
- Décrire le **comment** dans `/speckit.plan`, avec stack et architecture.
- Découper en tâches concrètes dans `/speckit.tasks`.
- Lancer l'exécution dans `/speckit.implement` seulement quand les étapes précédentes sont prêtes.

### Éviter les dérives pendant l'implémentation

Une **dérive** apparaît quand l'agent s'éloigne de la spec, ajoute des comportements non demandés ou modifie l'architecture sans autorisation. Spec Kit réduit ce risque en imposant des artefacts successifs et en faisant converger l'implémentation vers la spec, le plan et les tâches.

1. Vérifie que chaque tâche correspond à une exigence écrite.
2. Refuse les ajouts qui n'ont pas de lien direct avec la spec.
3. Demande une clarification avant d'élargir le périmètre.
4. Réexamine régulièrement les fichiers de référence avant de continuer.

### Utiliser la clarification comme garde-fou

La commande `/speckit.clarify` sert à lever les ambiguïtés avant le plan. C'est important, car une phrase floue dans la spec devient souvent une erreur de code plus tard. Le workflow recommandé par plusieurs sources inclut d'abord la clarification, puis la validation, puis le plan.

Un bon réflexe consiste à reformuler les zones sensibles en questions fermées :

- “La fonctionnalité doit-elle accepter plusieurs formats ?”
- “Cette règle est-elle bloquante ou informative ?”
- “Le comportement attendu en cas d’erreur est-il défini ?”

## Maintenir l’alignement avec les fichiers sources

Spec Kit est conçu pour fournir à l’agent un **contexte structuré**, pas une suite de messages isolés. La documentation GitHub précise que chaque artefact alimente le suivant, ce qui limite les prompts ad hoc et garde l’agent orienté vers les fichiers de référence.

- Utilise les fichiers générés comme source de vérité.
- Évite de réexpliquer toute la logique à chaque requête.
- Relis la spec et le plan avant de demander une modification.
- Pendant l’implémentation, demande des changements *localisés* plutôt qu’une réécriture globale.

## Exemple simple de cadrage

Si la spec dit “l’utilisateur peut exporter ses données”, l’agent ne doit pas décider seul du format, de l’interface ou des permissions. Ces choix doivent apparaître dans le plan ou être clarifiés avant l’implémentation. C’est exactement l’intérêt du développement piloté par la spécification : séparer le besoin, la décision technique et l’exécution.

## 36. Suivre l'avancement dans le projet

### Suivre l'avancement dans le projet

Suivre l'avancement, c'est **vérifier en continu où en est l'agent IA** par rapport aux tâches prévues, sans attendre la fin du codage. Dans Spec Kit, ce suivi s'appuie sur les artefacts générés avant l'implémentation, surtout `tasks.md`, qui sert de référence opérationnelle .

### Pourquoi ce suivi est indispensable

L'agent ne doit pas “avancer au feeling” : il doit exécuter les tâches **dans l'ordre des dépendances**, en respectant les repères de parallélisme indiqués par [P] quand ils existent. Cela réduit les écarts entre ce qui a été spécifié et ce qui est réellement codé, ce qui est précisément l'objectif d'un développement piloté par la spec.

Une bonne pratique consiste à considérer chaque tâche comme une preuve observable : soit elle est terminée avec les critères attendus, soit elle ne l'est pas encore.

### Ce qu'il faut surveiller concrètement

Pendant `/speckit.implement`, l'agent commence par vérifier que les fichiers nécessaires existent, notamment `constitution.md`, `spec.md`, `plan.md` et `tasks.md`. Ensuite, il traite les tâches dans l'ordre défini, ce qui permet de suivre la progression de façon simple : tâche en attente, tâche en cours, tâche terminée .

- **Dépendances** : une tâche ne doit pas démarrer si une autre n'est pas finie.
- **Marqueurs [P]** : les tâches parallèles peuvent avancer en même temps.
- **Critères de sortie** : une tâche n'est validée que si le résultat correspond à la spec.
- **Cohérence globale** : l'ensemble doit rester aligné avec les règles du projet et le plan technique.

### Exemple de suivi utile

Si `tasks.md` contient trois tâches, dont la première prépare un modèle de données et la deuxième construit l'API qui en dépend, l'agent doit terminer la première avant de lancer la seconde. En pratique, cela donne un suivi lisible : *préparation terminée, API en cours, tests à venir*.

- [x] Tâche 1 : schéma de données
- [ ] Tâche 2 : endpoint de création
- [ ] Tâche 3 : tests de validation

Ce type de vue est simple, mais il évite une dérive fréquente : coder vite sans pouvoir relier chaque modification à une exigence précise.

## Bonnes pratiques pour rester aligné

- **Relire l'état de la tâche avant chaque modification.**
- **Comparer le code produit avec `tasks.md` et `spec.md`.**
- **Signaler toute ambiguïté immédiatement** plutôt que de la contourner.
- **Ne valider une tâche que si son résultat est testable.**

Dans un projet piloté par Spec Kit, l'avancement n'est pas seulement une question de "pourcentage terminé" : c'est une lecture structurée du travail réel de l'agent, fichier par fichier, tâche par tâche, avec une traçabilité claire entre intention, plan et code .

## 37. Vérifier implémentation contre spec et plan

### Vérifier l'alignement

La vérification consiste à comparer l'**implémentation réelle** avec la **spec** et le **plan** pour détecter les écarts avant la mise en production. Spec Kit décrit explicitement une phase de **convergence** où l'on contrôle l'implémentation par rapport à la spec, au plan et aux tâches, jusqu'à obtenir l'état **Converged**.

Cette étape répond à une logique simple : *ce qui a été codé doit correspondre à ce qui a été demandé*. Sans ce contrôle, l'agent peut produire du code fonctionnel mais hors sujet, ou ignorer une contrainte d'architecture.

### Comparer les artefacts

Le contrôle s'appuie sur trois documents de référence : **spec.md** pour le *quoi*, **plan.md** pour le *comment*, et **tasks.md** pour l'exécution séquencée. GitHub Spec Kit indique que l'implémentation doit respecter ces artefacts dans l'ordre prévu.

Pour vérifier correctement, relis chaque exigence et cherche sa trace dans le code :

- chaque fonctionnalité demandée existe-t-elle vraiment ?
- chaque choix technique du plan a-t-il été appliqué ?
- chaque tâche est-elle terminée, sans contournement ?
- chaque dépendance, contrainte ou limite a-t-elle été respectée ?

### Repérer les écarts

Un écart fréquent est l'**implémentation partielle** : le code compile, mais une partie de la spec n'est pas couverte. Un autre écart courant est la **dérive de plan**, quand l'agent choisit une solution différente de celle validée dans plan.md.

Le test le plus utile est de relier chaque affirmation du cahier des charges à une preuve concrète dans le dépôt. Par exemple, si la spec exige une validation côté interface, il faut trouver le composant, la règle de validation et le test associé, pas seulement une intention dans un commentaire.

### Vérifier la qualité

Spec Kit met aussi en avant des **checklists qualité** avant ou pendant l'implémentation, notamment pour l'UX, les tests et la sécurité. Si ces contrôles restent incomplets, l'outil peut demander une confirmation avant de continuer.

Une vérification solide inclut donc :

- des **tests** alignés sur les cas d’usage décrits
- des **contraintes de sécurité** visibles dans le code ou la config
- une **architecture** cohérente avec le plan
- des critères d’acceptation vérifiables, pas seulement déclaratifs

## Conduire la convergence

La convergence n’est pas une simple relecture finale : c’est une boucle de correction. GitHub Spec Kit recommande de répéter l’implémentation et la vérification jusqu’à ce que la sortie indique **Converged**.

Concrètement, on corrige les écarts, puis on revérifie. Si une tâche a été mal interprétée, on revient au texte source plutôt que d’ajouter du code “pratique” mais non prévu. Cette discipline réduit les retours en review et les erreurs découvertes trop tard.

Le bon réflexe est de demander : *“Quelle ligne de la spec ou du plan justifie ce changement ?”* Si la réponse est floue, l’implémentation n’est pas encore validée.

### Mini-exemple de contrôle

- Spec : “authentification par email et mot de passe”
- Plan : “Next.js 14, TypeScript, validation serveur”
- Code attendu : formulaire, validation, gestion des erreurs, tests
- Code non conforme : formulaire sans validation, ou auth déportée dans un autre service non prévu



## 38. Répéter implémentation jusqu'à convergence

### Répéter jusqu'à convergence

Dans Spec-Kit, la **convergence** signifie que le code correspond réellement à la spec, au plan et à la liste des tâches. Le cycle n'est pas linéaire : après `/speckit.implement`, on exécute `/speckit.converge`, puis on recommence l'implémentation si des écarts restent ouverts.

L'idée est simple : on ne “termine” pas parce que du code existe, on termine quand l'agent a vérifié qu'il ne manque plus rien.

### Ce que fait `/speckit.converge`

La commande `/speckit.converge` compare le codebase aux artefacts du projet : `spec.md`, `plan.md` et `tasks.md`. Si elle détecte des lacunes, elle ajoute de nouvelles tâches dans `tasks.md` au lieu de laisser le problème invisible.

Concrètement, cela sert à repérer trois types d'écarts :

- une fonctionnalité prévue mais non codée ;
- une règle technique du plan non respectée ;
- une tâche marquée comme faite alors qu'un détail reste incomplet.

### Comment fonctionne la boucle

Le flux recommandé est : **implémenter** → **converger** → **réimplémenter** jusqu'à obtenir l'état *Converged*.

1. L'agent exécute les tâches dans l'ordre de dépendance avec `/speckit.implement`.
2. `/speckit.converge` vérifie la complétude face à la spec, au plan et aux tâches.
3. Si des trous sont trouvés, de nouvelles tâches sont ajoutées.
4. On relance `/speckit.implement` sur ces tâches.
5. On répète le cycle jusqu'à ce qu'aucun écart ne soit détecté.

Cette logique évite le faux sentiment de “fini” quand une fonctionnalité compile mais ne respecte pas encore toutes les exigences.

### Exemple pratique

Imaginons une API de réservation avec une spec qui exige :

- validation d'email ;
- gestion des doublons ;

- message d'erreur explicite.

L'implémentation initiale peut créer l'API et la base de données, mais oublier le message d'erreur. Dans ce cas, `/speckit.converge` signale l'écart et ajoute une tâche, par exemple : *ajouter un message d'erreur normalisé pour les doublons*.

On relance ensuite `/speckit.implement`, l'agent corrige, puis on reconverge. Ce va-et-vient continue jusqu'à ce que la vérification ne retourne plus de tâche nouvelle.

## Bonnes pratiques pour éviter les dérives

- Relire les nouvelles tâches avant de les coder pour confirmer qu'elles viennent bien d'un écart réel.
- Garder la spec stable pendant la boucle, sinon on mélange correction et changement de périmètre.
- Traiter chaque tâche ajoutée comme une exigence de qualité, pas comme un simple "bug mineur".

Dans la pratique, cette répétition est la partie qui transforme Spec-Kit en méthode de qualité : l'agent ne se contente pas de produire du code, il affine jusqu'à l'alignement complet avec le besoin exprimé.

## 39. Ajouter garde-fous architecture et CI

### Définir les garde-fous architecturaux

Les **garde-fous d'architecture** sont des règles qui empêchent un changement de casser la structure du logiciel. Un référentiel *Quality & Security by Design* regroupe précisément les règles d'architecture, de tests, de sécurité et d'acceptation qu'un changement doit respecter avant fusion, qu'il soit écrit par un humain, un assistant ou un agent IA.

Concrètement, ils protègent les **frontières entre modules**, les **dépendances autorisées**, la **duplication tolérée** et les **invariants** du système à préserver. Dans un projet Spec-Kit, cela veut dire que la spécification ne décrit pas seulement *quoi* construire, mais aussi *ce qui est interdit* .

### Poser des règles vérifiables

Un bon garde-fou doit être **testable**. Une règle vague comme “le code doit rester propre” ne suffit pas ; une règle utile dit par exemple : “le module **paiement** ne peut importer que **core** et **infra**”. Cette forme permet à la CI de vérifier automatiquement le respect de la règle.

Vous pouvez structurer ces règles en trois familles :

- **Architecture** : imports autorisés, couches, dépendances cycliques interdites.
- **Qualité** : taille maximale d'une PR, complexité acceptable, duplication limitée.
- **Sécurité** : secrets interdits, accès aux données sensibles restreints .

Un garde-fou utile doit être *observable*, sinon il reste une intention et non un contrôle.

### Brancher les garde-fous à la CI

La **CI** (*intégration continue*) exécute automatiquement les vérifications à chaque modification. Son rôle est d'empêcher la fusion d'un changement non conforme. Les guides sur les garde-fous pour agents IA insistent sur des contrôles comme les listes blanches d'actions, les portes d'approbation et le mode simulation, ainsi que sur le test explicite de ces protections.

Dans la pratique, la CI peut lancer :

1. une analyse des dépendances entre modules ;
2. des tests unitaires et d'acceptation ;
3. une vérification des politiques de sécurité ;
4. un contrôle du format des réponses si l'IA produit du code ou des artefacts structurés.

## Exemple simple de règle CI

*# exemple de contrôle CI*

checks:

- name: no forbidden import  
run: python scripts/check\_imports.py
- name: architecture boundaries  
run: python scripts/check\_architecture.py
- name: approval required for destructive action  
run: python scripts/check\_approvals.py

Ce type de pipeline transforme la spécification en **contrat exécutable**. Si un agent IA propose une modification qui touche une zone interdite, la CI bloque la fusion avant que le problème n'atteigne la branche principale.

## Tester les garde-fous eux-mêmes

Un garde-fou non testé est une fausse sécurité. Les sources sur les garde-fous IA recommandent de tester les attaques indirectes, les scénarios en plusieurs étapes et les chemins sûrs pour vérifier que la protection se déclenche bien sans bloquer inutilement le travail légitime.

Vous pouvez ajouter des tests comme :

- une tentative d'import interdit ;
- une action destructrice simulée ;
- une modification qui franchit une frontière de module ;
- une demande légitime qui doit passer sans approbation inutile .

Cette approche est particulièrement adaptée à Spec-Kit : la spécification définit la règle, puis la CI prouve qu'elle reste vraie à chaque changement.

## 40. Documenter la livraison finale

### Rédiger le dossier de livraison finale

La **livraison finale** est l'ensemble des documents qui prouvent ce qui a été construit, comment cela a été validé, et dans quel état le produit peut être remis. Dans Spec Kit, le flux **Spec** → **Plan** → **Tasks** → **Implement** sert de base, puis la livraison rassemble les artefacts utiles pour la suite du projet.

Un bon dossier de livraison ne réécrit pas le code : il **trace** les décisions, les résultats de validation et les limites connues. Cette logique est cohérente avec le développement fondé sur les spécifications, où les changements restent reliés à l'intention d'affaires et à la vérification qualité.

### Ce que la livraison doit contenir

La documentation finale doit être **vérifiable**, **reproductible** et **liée aux livrables produits**. Les sources sur Spec Kit mentionnent des documents complémentaires comme les contrats d'API, les scénarios de test et un guide de validation rapide, utiles pour prouver que l'implémentation correspond bien à la spec.

- la **spécification validée** ;
- le **plan technique** ;
- la **liste des tâches réalisées** ;
- les **scénarios de test exécutés** ;
- les **écarts connus** et leurs impacts ;
- les **instructions de validation** pour relancer le projet.

Exemple simple : si une fonctionnalité de connexion a été livrée, le dossier final doit préciser le comportement attendu, les cas testés, les erreurs observées, et la manière de vérifier le résultat sans ambiguïté.

### Décrire les résultats, pas seulement le code

La documentation de livraison doit raconter le **résultat concret** du travail. Cela inclut les fonctionnalités effectivement présentes, les limites acceptées, et les points à surveiller lors de la mise en service. Les sources sur la QA avec Spec Kit rappellent que les tests d'acceptance doivent exister dès le départ, car ils ne peuvent pas être “rétro-générés” proprement depuis le code une fois le travail terminé.

Pour être utile, le document final doit répondre à trois questions :

1. **Qu'est-ce qui a été livré ?**

2. **Comment a-t-on vérifié que cela fonctionne ?**
3. **Qu'est-ce qui reste ouvert ou différé ?**

Cette structure évite les livraisons floues et facilite la relecture humaine avant acceptation finale.

## Relier la livraison aux preuves de qualité

Une livraison finale solide doit montrer que le produit a été contrôlé par rapport à la spec, au plan et aux tâches. Spec Kit prévoit justement une étape de convergence qui vérifie la complétude du codebase face aux documents de référence, et qui peut ajouter des tâches si des écarts sont détectés.

Dans le dossier final, ajoutez :

- les **résultats de tests** ;
- les **incertitudes levées** pendant l'implémentation ;
- les **points de conformité** ;
- les **écarts documentés** avec justification ;
- les **éléments à revérifier** avant mise en production.

Cette manière de documenter est proche des exigences qualité industrielles, où une livraison doit être accompagnée des informations documentées requises pour être acceptée.

## Écrire une livraison exploitable par l'équipe

Une bonne documentation finale sert aussi au prochain développeur. Elle doit expliquer ce qui est stable, ce qui dépend d'un contexte précis, et comment reprendre la main sans deviner. Les guides Spec Kit insistent sur la clarté, la validation des exigences et la production d'artefacts durables, reliés entre eux.

Pour rester exploitable, utilisez un format simple :

- un **résumé fonctionnel** en langage clair ;
- un **état de validation** avec faits observables ;
- une **liste des fichiers livrés** ;
- des **notes de reprise** pour les cas limites ;
- une **trace de décision** pour les arbitrages importants.

Un exemple utile est une note du type : *“Le mode export CSV est livré, validé sur 3 jeux de données, mais la pagination serveur reste hors périmètre.”* Ce niveau de précision rend la livraison finale directement réutilisable et limite les malentendus.

## 41. Installer l'extension de correction de bugs

### Installer l'extension de correction de bugs

L'extension **bug** de Spec Kit s'installe volontairement, puis elle ajoute un flux fiable en trois étapes : **assess** → **fix** → **test**. Elle sert à traiter un bug de manière traçable, avec un dossier dédié dans `.specify/bugs/` pour conserver les preuves et la résolution.

### Préparer l'environnement

Avant l'installation, ouvrez votre projet dans le répertoire où votre agent IA de code est déjà configuré. Spec Kit détecte l'agent présent en repérant des dossiers comme `.github/prompts/` ou `.claude/commands/`, puis place les commandes au bon endroit.

Vous pouvez vérifier ensuite que l'outillage répond bien avec :

```
specify extension list
```

Cette vérification confirme si l'extension est installée et activée. Si elle apparaît comme installée mais ne réagit pas, un redémarrage de l'agent IA peut être nécessaire pour recharger les commandes.

### Installer l'extension bug

Pour installer l'extension intégrée de correction de bugs, la commande recommandée est :

```
specify extension add bug
```

Spec Kit indique explicitement que cette extension est *bundled* mais *opt-in* : elle n'est pas installée automatiquement, donc il faut la demander.

Après l'installation, Spec Kit copie les fichiers de commande dans le dossier adapté à l'agent utilisé, met les mémoires dans `.specify/memory/`, les gabarits dans `.specify/templates/`, et consigne l'ajout dans `.specify/extensions.json`. Ce suivi permet de savoir exactement quelle extension a été ajoutée et quand.

### Utiliser les commandes installées

Une fois l'extension active, le flux de travail devient simple et reproductible. Le dépôt officiel décrit trois commandes principales, exécutées dans le répertoire du projet :

1. `/speckit-bug-assess "<bug report>" slug=login-crash`
2. `/speckit-bug-fix slug=login-crash`
3. `/speckit-bug-test slug=login-crash`

Chaque bug est isolé par un **slug**, c'est-à-dire un identifiant court qui évite de mélanger plusieurs incidents. Cette séparation rend l'analyse plus nette et facilite le suivi du correctif.

## Vérifier et dépanner l'installation

Après installation, relancez `specify extension list` pour confirmer l'état réel de l'extension, puis redémarrez l'agent si les nouvelles commandes n'apparaissent pas. La documentation précise que l'agent peut avoir besoin d'être rechargé pour prendre l'extension en compte.

Si la commande n'est toujours pas disponible, vérifiez que vous utilisez bien la CLI Spec Kit officielle. Des problèmes de version ont déjà provoqué l'absence de la commande **extension** dans certaines installations, ce qui bloque l'ajout des extensions.

Exemple pratique : si vous avez installé **bug** mais que `/speckit-bug-assess` n'est pas reconnu, la séquence la plus utile est `specify extension list`, puis redémarrage de l'agent, puis réinstallation si nécessaire.

## Bon réflexe de mise en place

Installez l'extension dans le projet concerné, pas "en théorie" dans un autre espace, afin que les commandes, les fichiers mémoire et les modèles restent cohérents avec le dépôt réel. Cette proximité entre le code, les commandes et le dossier `.specify/bugs/` est ce qui rend la correction plus fiable.



## 42. Évaluer un rapport et isoler la cause

### Lire le rapport sans supposer la cause

Évaluer un rapport consiste à distinguer le **symptôme observé** de la **cause réelle**. Dans Spec-Kit, le réflexe utile est de ne pas passer directement au correctif, car une correction prématurée peut masquer le vrai problème et créer un faux sentiment de résolution.

Un rapport de bug est un *indice*, pas une preuve. Il peut décrire un plantage, un message d'erreur, une régression ou un comportement inattendu, mais la tâche consiste d'abord à vérifier si le signal décrit correspond bien à un bug réel et reproductible.

- Identifier le message exact, l'écran touché ou la fonction concernée.
- Noter la version, l'environnement et les étapes de reproduction.
- Séparer ce qui est *observé* de ce qui est *interprété*.

### Extraire les faits utiles du rapport

L'analyse commence par les éléments concrets : entrées utilisateur, sortie du système, fréquence du problème et conditions d'apparition. Cette discipline évite de confondre une corrélation avec une causalité, surtout quand un agent d'IA reçoit un long récit ambigu.

Une bonne évaluation transforme un récit flou en faits vérifiables.

Dans la pratique, on cherche trois choses : **le déclencheur**, **la zone de code suspecte** et **la trace de reproduction**. Spec-Kit recommande un flux *assess* → *fix* → *test*, précisément pour garder le diagnostic lié à des preuves et documenter ce raisonnement étape par étape.

### Isoler la cause probable

Isoler la cause, c'est réduire le champ des hypothèses jusqu'à trouver l'élément qui explique le symptôme avec le moins d'ambiguïté possible. Cela peut être une mauvaise gestion d'état, une entrée non validée, une dépendance cassée ou un problème de contexte dans un workflow agentique.

Un exemple fréquent est le rapport qui semble pointer le dernier changement, alors que la vraie origine est une interaction entre plusieurs étapes ou entre l'outil et son contexte d'exécution. Les retours sur Spec-Kit signalent justement que les workflows multi-étapes peuvent devenir fragiles quand le contexte sature ou quand plusieurs agents partagent le même état local.

- Vérifier si le bug apparaît sur une branche propre.
- Comparer avec une version antérieure connue comme saine.

- Repérer les fichiers ou modules touchés par la reproduction.
- Tester l'hypothèse la plus simple avant les scénarios complexes.

## Formuler un diagnostic traçable

Un bon diagnostic doit être *traçable* : chaque affirmation doit pouvoir remonter à une observation précise. Dans la documentation Spec-Kit, l'étape d'évaluation produit un rapport dédié et prépare une remédiation fondée sur des preuves, pas sur une intuition.

Exemple utile : si un utilisateur signale que « la commande ne marche plus », le diagnostic doit préciser si la commande échoue partout, seulement dans un environnement, ou seulement après une mise à jour. Cette distinction change complètement l'orientation de l'analyse.

- Symptôme : ce que l'utilisateur voit.
- Cause probable : l'explication la plus plausible.
- Preuve : le log, la reproduction ou la différence observée.
- Incertitude : ce qui reste à confirmer avant de corriger.

## Savoir quand le rapport est insuffisant

Parfois, le rapport ne permet pas d'isoler la cause. Il manque alors une version, un log, une reproduction minimale ou un détail d'environnement. Dans ce cas, l'objectif n'est pas d'inventer une explication, mais de demander l'information qui permet de réduire l'incertitude.

Un rapport est exploitable quand il permet de répondre à ces questions simples :

- Le problème est-il reproductible ?
- Quelle action le déclenche ?
- Quel résultat était attendu ?
- Quel résultat exact a été obtenu ?

Quand ces éléments sont présents, l'analyse peut avancer de manière fiable et le diagnostic reste solide, mesurable et réutilisable.

## 43. Appliquer un correctif ciblé et tracé

### Cadrer le correctif avant de coder

Un **correctif ciblé et tracé** consiste à modifier *le minimum nécessaire* pour supprimer un bug, tout en gardant une trace claire de ce qui a changé et pourquoi. Dans Spec-Kit, cette discipline réduit le risque d'introduire un nouveau défaut en corrigeant l'ancien, ce qui est précisément le danger signalé quand on passe trop vite du rapport au patch.

Le bon réflexe est de partir de la cause déjà établie, puis d'appliquer une réparation étroite, vérifiable et documentée. Le flux recommandé par Spec Kit pour les bugs suit une séquence simple : **assess** → **fix** → **test**.

### Définir un périmètre strict

Un correctif *ciblé* ne touche qu'un point de code lié à la cause racine. Si le bug vient d'une condition mal gérée, on corrige cette condition, pas toute la fonction. Cette approche évite les changements larges qui compliquent la revue et brouillent l'origine de la réparation.

Pour rester précis, il faut écrire l'intention du patch avant l'édition du code :

- quelle cause est corrigée ;
- quel fichier ou quelle fonction est concerné ;
- quel comportement attendu doit réapparaître ;
- quelle partie du système ne doit pas changer.

Un bon correctif ressemble à un scalpel, pas à une chirurgie lourde

### Tracer chaque modification

Le mot **tracé** signifie ici que la correction doit pouvoir être reliée à l'incident initial, à la cause identifiée et au résultat attendu. Spec Kit encourage justement une chaîne lisible entre l'évaluation, la correction et le test, afin que chaque étape reste auditable.

Concrètement, cela veut dire :

- associer le patch à un identifiant de bug, comme un *slug* ;
- conserver le lien entre diagnostic et changement ;
- noter la justification du choix technique ;
- éviter les modifications “bonus” non demandées.

Cette traçabilité est utile quand plusieurs agents ou plusieurs sessions interviennent sur le même dépôt. Elle permet de distinguer une vraie correction d'un simple ajustement opportuniste.

## Exemple de patch minimal

Si un login plante quand un champ est vide, le correctif ciblé ne réécrit pas tout le formulaire. Il ajoute seulement la vérification manquante à l'endroit exact où l'entrée est utilisée.

```
function login(user) {  
  if (!user || !user.email) {  
    throw new Error("Email requis");  
  }  
  return authenticate(user.email);  
}
```

Ici, le changement est limité à la validation de l'entrée. Le code est aussi plus facile à relier au bug initial, car la cause et la réparation se correspondent directement.

## Garder le lien avec la preuve

Spec Kit recommande de valider ensuite que la correction résout bien le symptôme d'origine, sans casser le reste du comportement. Même si le test appartient à l'étape suivante, le correctif doit déjà être conçu pour être testable, sinon il devient impossible à relier proprement à la preuve de réussite.

Pour rester rigoureux, vérifie toujours :

- la ligne ou la branche de code réellement fautive ;
- le comportement corrigé observé en reproduction ;
- l'absence d'effets secondaires visibles ;
- la capacité à expliquer le patch en une phrase claire.

Un patch bien tracé peut être compris après coup par un autre développeur, car il raconte une histoire complète : bug, cause, intervention, résultat. Cela rend la maintenance plus sûre et la correction plus crédible

## 44. Tester et valider la résolution

### Vérifier d’abord la correction réelle

Après un correctif, la priorité est de confirmer que le bug a disparu *sans casser le reste*. Dans Spec-Kit, la validation ne se limite pas à “le code compile” : elle doit montrer que le comportement attendu est bien revenu et que les critères de réussite sont atteints.

Une résolution n’est fiable que si elle est observable par un test reproductible, pas seulement par une impression de “ça a l’air bon”.

### Choisir les bons tests

La validation commence par le test qui échoue avant le correctif, puis passe au test qui réussit après. Cette logique ressemble à une boucle simple : reproduire, corriger, re-tester. C’est aussi l’esprit de l’approche guidée par spécifications, où chaque étape doit être validée avant de continuer.

- **Test de reproduction** : il rejoue précisément le scénario du bug.
- **Test de non-régression** : il vérifie qu’aucun comportement existant n’a été détruit.
- **Test d’acceptation** : il confirme que le besoin exprimé dans la spécification est satisfait.

Un bon exemple : si un bouton crashait quand le champ est vide, le test doit couvrir *champ vide*, *champ rempli*, puis *retour normal* après correction.

### Valider avec les artefacts Spec-Kit

Spec-Kit insiste sur la cohérence entre spécification, plan, tâches et implémentation. La validation doit donc relire ces artefacts ensemble pour vérifier qu’aucune exigence n’a été oubliée et qu’aucune tâche ne dépasse le périmètre fixé.

1. Relancer les tests liés au bug.
2. Vérifier que les critères d’acceptation sont passants.
3. Contrôler les cas limites mentionnés dans la spécification.
4. Comparer le comportement observé avec le plan initial.

Cette étape évite les corrections “partielles”, où le bug principal disparaît mais un cas adjacent reste cassé.

## Contrôler les cas limites

La fiabilité vient souvent des cas que l'on oublie au premier passage. Les guides de validation recommandent d'examiner les entrées extrêmes, les données manquantes, les erreurs de format et les scénarios rares, car ce sont eux qui révèlent les faiblesses d'un correctif.

- Valeurs vides ou nulles
- Données très longues
- Réponses réseau lentes ou absentes
- Utilisation répétée de la même action

Par exemple, un correctif sur une recherche peut réussir sur une requête simple mais échouer sur une chaîne vide ou un caractère spécial.

## Documenter la preuve de résolution

Une validation sérieuse produit une preuve claire : test initial en échec, correctif appliqué, test final en réussite. Dans les démarches de développement piloté par spécifications, cette preuve aide à garder un historique lisible et à confirmer que la solution répond bien au problème défini.

Le plus utile est de conserver :

- le scénario de reproduction
- le résultat avant correction
- le résultat après correction
- la liste des tests de non-régression exécutés

Cette trace rend la résolution vérifiable par une autre personne, ce qui est essentiel pour une correction fiable.

## 45. Installer l'extension d'évaluation d'idées

### Installer l'extension d'évaluation d'idées

L'**extension d'évaluation d'idées** de Spec Kit s'installe avec la CLI **specify**, comme les autres extensions du catalogue communautaire. GitHub indique que les extensions ajoutent des capacités nouvelles et qu'elles peuvent être installées, mises à jour, activées ou supprimées indépendamment.

Pour commencer, vérifiez que **Spec Kit** est déjà initialisé dans votre projet, puis utilisez la commande d'installation adaptée. La documentation officielle montre la forme générale **specify extension add <name>**, et précise que l'installation peut venir du catalogue, d'une URL ou d'un dossier local.

### Commande d'installation

La manière la plus simple consiste à chercher d'abord l'extension, puis à l'ajouter. La commande de recherche est **specify extension search**, et l'installation est **specify extension add <name>**. Cette logique est décrite dans la documentation des extensions Spec Kit.

```
specify extension search
specify extension add <name>
specify extension list
```

Si vous travaillez sur une extension locale ou non publiée, la documentation prévoit **--dev** pour un dossier local, et **--from <url>** pour une source personnalisée. Elle mentionne aussi **--force** pour écraser une installation existante.

### Vérifier que l'extension est active

Après l'installation, utilisez **specify extension list** pour confirmer que l'extension apparaît bien dans la liste. La documentation précise que cette commande affiche l'état, la version et le nombre de commandes fournies par l'extension.

Si l'extension est installée mais ne semble pas fonctionner dans votre agent IA, la documentation recommande de redémarrer l'agent pour qu'il recharge les commandes.

C'est important, car Spec Kit enregistre automatiquement les commandes d'extension dans l'intégration de l'agent IA déjà installée. En pratique, cela signifie que l'installation ne suffit pas toujours : il faut parfois relancer l'environnement pour voir apparaître les nouveaux slash commands.

## Configurer l’extension

Certaines extensions nécessitent un fichier de configuration copié depuis un modèle. La documentation indique un schéma standard dans `.specify/extensions/<ext>/`, avec un fichier template à dupliquer vers un fichier de configuration actif.

```
cp .specify/extensions/<ext>/<ext>-config.template.yml \
.specify/extensions/<ext>/<ext>-config.yml
```

Cette étape est utile quand l’extension d’évaluation d’idées demande des paramètres comme des critères, des seuils ou des options de sortie. Le principe est simple : le *template* est le brouillon fourni par l’auteur, et le fichier `-config.yml` est votre version personnalisée.

## Exemple d’usage concret

Une fois l’extension installée, vous pouvez l’utiliser pour structurer une revue d’idée avant codage. Par exemple, si votre extension ajoute une commande de validation, elle peut apparaître comme une nouvelle commande disponible dans votre session d’agent, au même titre que les commandes natives de Spec Kit.

- Rechercher l’extension dans le catalogue
- L’installer avec `specify extension add`
- Vérifier avec `specify extension list`
- Copier le fichier de configuration s’il existe
- Redémarrer l’agent si nécessaire

Cette approche évite d’ajouter des outils “à la main” dans le projet. Spec Kit centralise ainsi le déploiement, l’activation et la maintenance des extensions dans un flux unique, ce qui rend l’évaluation d’une idée plus reproductible et plus simple à partager au sein d’une équipe.



## 46. Collecter et rechercher preuves pour contre

### Pourquoi chercher des preuves pour et contre

Dans une idée de logiciel IA, la phase de collecte de preuves sert à éviter de construire une solution séduisante mais fragile. Elle consiste à réunir des éléments *favorables* à l'idée et des éléments *contraires* pour tester sa solidité avant d'écrire le moindre code.

Le principe est simple : une hypothèse n'est pas une décision. Une hypothèse devient crédible seulement si elle résiste à des preuves opposées, comme un dossier de justice qui doit être corroboré par plusieurs sources indépendantes.

### Ce qu'on appelle une preuve utile

Une **preuve utile** est un fait observé, daté, vérifiable et relié à un problème concret. En pratique, cela peut être un retour utilisateur, un log d'usage, un prototype testé, ou un signal terrain mesuré de façon traçable.

Pour garder des preuves fiables, il faut privilégier :

- des sources identifiables ;
- des données datées ;
- des éléments reproductibles ;
- des traces comparables entre plusieurs sources.

L'idée n'est pas de prouver que l'idée est "bonne", mais de vérifier si elle est *réellement défendable* face aux objections. C'est exactement ce que recommandent les démarches qui insistent sur l'intégrité, la traçabilité et la corroboration des éléments.

### Méthode pour collecter les preuves pour

Commencez par formuler l'hypothèse en une phrase : "Cette fonctionnalité réduit le temps de traitement de 30 %" ou "Ce marché a un besoin non satisfait". Puis cherchez des preuves qui vont dans ce sens.

Exemples de preuves favorables :

- entretiens où plusieurs personnes décrivent le même besoin ;
- données internes montrant des tâches répétitives ;
- demandes récurrentes du support ;
- prototypes validés par des tests d'usage.

Dans l'IA, une preuve favorable peut aussi être un signal de faisabilité : qualité suffisante des données, stabilité du workflow, ou capacité à identifier des cas d'usage bien cadrés. Plus

les faits sont précis, plus l'argument est exploitable.

## Méthode pour collecter les preuves contre

Les preuves contraires sont souvent les plus utiles, car elles révèlent les limites cachées. Cherchez activement ce qui invaliderait l'idée : coût trop élevé, données absentes, utilisateur non motivé, risque juridique, ou performance trop faible.

Sources de contre-preuves à examiner :

- cas d'usage déjà abandonnés ;
- retours négatifs d'utilisateurs cibles ;
- métriques montrant une faible fréquence du problème ;
- contraintes de conformité ou d'authenticité des données.

Une bonne règle pratique : si vous ne trouvez aucune preuve contre, c'est probablement que vous n'avez pas encore assez cherché.

## Organiser la confrontation des preuves

Pour comparer proprement, classez chaque preuve selon trois questions :

1. Que démontre-t-elle exactement ?
2. Quelle est sa fiabilité ?
3. Que manque-t-il pour la contredire ou la confirmer ?

Cette logique évite les conclusions rapides. Dans les contextes où l'IA produit des contenus ou des signaux ambigus, la valeur d'un élément dépend fortement de son authenticité, de sa traçabilité et de sa vérification croisée.

Exemple simple : si trois utilisateurs demandent une fonctionnalité, mais que les logs montrent qu'ils n'utilisent pas la version actuelle, la demande est réelle mais peut ne pas justifier un développement immédiat.

## Rédiger une décision fondée sur les preuves

Une bonne décision finale doit distinguer *preuves fortes*, *preuves faibles* et *zones inconnues*. Cette séparation aide à décider “go” ou “stop” sans confondre intuition et observation.

Structurez votre note ainsi :

- **ce qui soutient l'idée ;**
- **ce qui la contredit ;**
- **ce qui reste à vérifier ;**

- **le niveau de confiance actuel.**

Cette discipline transforme une idée en objet d'enquête. Elle réduit les biais, oblige à chercher des faits opposés, et rend la suite du travail beaucoup plus sûre.

## 47. Définir problème objectifs et indicateurs

### Définir le problème

Dans Spec-Kit, cette étape sert à formuler le **vrai besoin** avant toute écriture de code. Le dépôt GitHub décrit explicitement une phase “Define” pour *the problem, goals, and success metrics* après l’intake et la recherche d’éléments favorables et défavorables.

Un *problème bien défini* répond à trois questions simples : **qui** est touché, **quoi** ne fonctionne pas, et **pourquoi** cela compte. Par exemple, “les utilisateurs abandonnent le panier” est plus utile que “l’application est mauvaise”, car la première phrase cible un phénomène observable.

### Formuler les objectifs

Un objectif décrit l’**effet attendu** du logiciel, pas la solution technique. Dans une logique de spécification, il faut séparer le *quoi* du *comment* : on cherche d’abord ce que le produit doit améliorer, puis seulement ensuite la manière de le construire.

Pour être exploitable, l’objectif doit rester concret. Les sources sur les KPIs IA recommandent de partir d’un objectif métier clair, comme gagner du temps, réduire les erreurs, améliorer la satisfaction ou fluidifier un processus, puis d’en dériver quelques indicateurs utiles.

Exemples d’objectifs bien formulés :

- Réduire le temps de traitement des demandes de support.
- Diminuer le taux d’erreurs sur une tâche répétitive.
- Augmenter le taux d’adoption d’une fonctionnalité IA.
- Améliorer la satisfaction client après interaction.

### Choisir les indicateurs

Un **indicateur** est une mesure qui permet de vérifier si l’objectif avance réellement. Les sources convergent sur une règle pratique : ne pas multiplier les métriques, mais retenir *3 à 5 KPI maximum* par cas d’usage.

Un bon indicateur doit être :

- **spécifique** : lié à un résultat précis ;
- **mesurable** : observable dans des données ;
- **stable** : comparable dans le temps ;
- **actionnable** : utile pour décider d’un ajustement.

Exemple simple : si l'objectif est de réduire les délais, l'indicateur principal peut être le **temps moyen de traitement**, complété par un indicateur d'adoption et un indicateur de qualité, afin d'éviter d'améliorer la vitesse au détriment du résultat.

## Fixer une base de référence

Avant de viser une amélioration, il faut mesurer la **baseline**, c'est-à-dire la situation de départ. Plusieurs sources insistent sur ce point : on compare ensuite l'après-déploiement à l'état initial pour savoir si le système apporte réellement de la valeur.

Cette base doit être simple à documenter :

- valeur actuelle de l'indicateur ;
- période de mesure ;
- source de données ;
- responsable du suivi.

Par exemple, si une équipe traite 120 demandes par jour avec 18 minutes de délai moyen, ces chiffres deviennent le point de départ. Sans eux, impossible de dire si le logiciel a amélioré la situation ou seulement déplacé le problème.

## Rédiger une définition exploitable

La meilleure pratique consiste à écrire une phrase courte qui relie problème, objectif et indicateur. Cette formulation sert de garde-fou pendant toute la suite du travail, car elle évite de dériver vers des fonctionnalités séduisantes mais hors sujet

Une structure utile :

- **Problème** : ce qui bloque aujourd'hui.
- **Objectif** : ce que l'on veut changer.
- **Indicateur** : comment on le vérifiera.
- **Cible** : à quel niveau le succès sera considéré comme atteint.

Exemple : “Les réponses de support prennent trop de temps ; nous voulons réduire le délai moyen de 30 % en trois mois, mesuré sur les tickets clos.”

## 48. Façonner options puis décider go ou stop

### Décider avant d'implémenter

Dans Spec-Kit, **façonner des options puis décider go ou stop** signifie qu'on ne lance pas le code trop tôt : on compare plusieurs solutions, on explicite leurs compromis, puis on tranche si l'idée mérite d'avancer, d'être clarifiée, ou d'être arrêtée. GitHub indique que le flux d'évaluation est autonome, avec une étape dédiée à la forme des solutions puis une étape de décision (`/speckit-assess-shape` puis `/speckit-assess-decide`).

L'objectif est simple : éviter qu'un agent d'IA transforme une idée floue en code irréversible.

### Ce que signifie “façonner des options”

**Façonner** veut dire transformer une intuition en alternatives concrètes. Au lieu de dire “fais une appli rapide”, on formule 2 ou 3 pistes : une version simple, une version robuste, une version économique. Chaque option doit préciser *ce qu'elle apporte*, *ce qu'elle coûte* et *ce qu'elle risque*. Cette logique rejoint les bonnes pratiques de spécification structurée, où l'on clarifie les hypothèses avant le plan technique.

Par exemple :

- Option A : prototype minimal, rapide à tester
- Option B : solution plus complète, mais plus longue à livrer
- Option C : abandon de la fonctionnalité si le coût dépasse le bénéfice

Cette étape sert à rendre visible le compromis, au lieu de le cacher dans le prompt de l'agent. GitHub recommande ensuite de passer à la décision seulement après cette comparaison structurée.

### Comment comparer les options

Pour comparer proprement, garde trois critères simples : **valeur**, **coût**, **risque**. La valeur mesure l'intérêt pour l'utilisateur ou le produit, le coût mesure l'effort de développement, et le risque mesure les incertitudes techniques ou fonctionnelles. Les guides autour de Spec-Kit insistent aussi sur la nécessité de signaler les zones ambiguës et les hypothèses non couvertes avant d'aller plus loin.

Tu peux te poser ces questions :

1. L'option répond-elle vraiment au besoin principal ?
2. L'IA a-t-elle dû inventer des détails non spécifiés ?

### 3. Faut-il demander une clarification humaine avant d'aller en planification ?

Un bon façonnage rend visibles les **hypothèses**. Si une option dépend d'une donnée inconnue, d'une API non confirmée ou d'une règle métier floue, elle doit être marquée comme fragile plutôt que présentée comme acquise.

## Quand dire go

Le verdict **go** signifie : l'idée est assez claire, assez utile et assez réaliste pour passer à la spécification détaillée ou à la planification. GitHub précise que si la décision est go, l'idée peut être transmise à l'étape suivante du processus Spec-Kit.

Tu peux dire go quand :

- le besoin est compréhensible sans interprétation lourde
- les options sont cohérentes avec les contraintes
- les risques majeurs sont connus et acceptables

Exemple : si une fonctionnalité “exporter un rapport PDF” peut être réalisée avec une solution simple, sans dépendance risquée, le go est justifié. Si, au contraire, l'idée demande une intégration incertaine ou des règles métier mal définies, le go serait prématuré.

## Quand dire stop

**Stop** ne veut pas dire “idée mauvaise”. Cela veut dire : pas assez solide pour coder maintenant. Les approches de spec-driven development recommandent d'arrêter quand les incertitudes dominent, quand les exigences restent ambiguës, ou quand une décision importante nécessite validation humaine avant tout engagement technique.

Le stop est pertinent si :

- les objectifs sont contradictoires
- le coût dépasse la valeur attendue
- une hypothèse critique n'a pas été vérifiée

Dans la pratique, stop protège ton projet : tu évites de construire une mauvaise solution avec beaucoup de vitesse. Cela rejoint l'idée centrale de Spec-Kit : l'agent doit travailler sur une base décidée, pas devinée.

## Une règle simple pour trancher

Prends une décision explicite à la fin de l'évaluation :

1. **Go** si l'option est claire, utile et suffisamment sûre.

2. **Clarify** si une information manque encore.
3. **Stop** si le coût, le risque ou l'ambiguïté restent trop élevés.

Cette discipline rend la suite plus fiable, car elle transforme une idée en choix argumenté avant toute écriture de code .



## 49. Créer presets extensions et workflows

### Définir le rôle des presets

Un **preset** est un paquet de configuration qui change la manière dont Spec Kit produit ses artefacts, sans modifier le cœur de l'outil. Il sert à imposer un style d'équipe, une terminologie métier ou une structure de livrables cohérente . Les presets peuvent remplacer des modèles et des commandes, tout en restant *stackables* et ordonnés par priorité.

Idée simple : une extension ajoute des capacités, tandis qu'un preset change la façon d'utiliser les capacités déjà présentes.

### Préparer un preset personnalisé

La méthode recommandée consiste à partir du dossier `presets/scaffold/`, puis à le copier dans un nouveau répertoire pour créer votre base locale. Ensuite, vous éditez `preset.yml` pour définir un identifiant unique, la version minimale de Spec Kit attendue, et les éléments qui doivent être remplacés.

Vous pouvez ensuite ajouter ou remplacer des fichiers dans `templates/` pour modifier les spécifications, plans, tâches ou checklists générés. Cette approche est utile quand vous voulez, par exemple, transformer le vocabulaire standard en termes métier propres à votre organisation, comme le montre un preset communautaire qui réécrit tout le flux en langage pirate sans toucher au moteur.

### Remplacer modèles et commandes

Les presets peuvent surcharger les fichiers de base et aussi ceux fournis par des extensions installées. Pour les templates, on utilise la règle de remplacement explicite indiquée dans le manifeste, afin de cibler précisément le fichier du cœur ou d'une extension.

Pour les commandes, un fichier Markdown placé dans `commands/` définit le comportement attendu de l'agent LLM. Selon le type d'agent, la syntaxe d'entrée peut utiliser `$ARGUMENTS` ou `{{args}}`, et le registraire interne convertit ensuite cette forme vers le format approprié. Cela permet de standardiser un flux, par exemple une commande de revue qui impose toujours le même format de sortie .

### Tester localement avant publication

Le bon réflexe est d'installer le preset en mode développement avec `specify preset add --dev .`, ce qui relie le dossier local sans emballage final. Ensuite, vérifiez la résolution

des templates avec `specify preset resolve spec-template`, afin de confirmer que votre version remplace bien celle du noyau.

Un test local est important parce que les presets sont *versionnés* et peuvent être combinés avec d'autres presets. La priorité détermine quel fichier gagne lorsqu'il y a conflit : le plus prioritaire l'emporte dans la résolution. Cela évite les surprises quand plusieurs équipes partagent les mêmes conventions.

## Combiner presets et extensions

Les **extensions** créent de nouvelles capacités, comme des workflows spécialisés, des intégrations externes ou des phases de développement supplémentaires. Un preset, lui, ne crée pas nécessairement du nouveau : il reformule, restreint ou harmonise ce qui existe déjà.

Cette séparation est pratique pour garder une architecture lisible. Par exemple, une extension peut ajouter un parcours métier complet, puis un preset peut imposer le vocabulaire, la structure des livrables et les règles de conformité associées. Dans la documentation communautaire, un preset de workflow ajoute même plusieurs modèles et commandes pour orchestrer des rôles d'agents, preuve que la frontière entre “étendre” et “personnaliser” reste complémentaire.

## 50. Explorer processus AIDE Canon Product Forge

### AIDE dans Spec Kit

Le processus **AIDE** de **Canon Product Forge** organise le travail en **boucle guidée** : on part d'une vision, puis on affine le plan, on exécute, et on vérifie avant de recommencer. Dans Spec Kit, cela s'inscrit dans une logique de développement piloté par les spécifications, avec un enchaînement clair et contrôlé.

**AIDE** signifie ici une progression opérationnelle qui relie la stratégie produit à l'exécution, plutôt qu'une simple suite de commandes. L'idée est de garder la spécification comme *source de vérité* tout au long du cycle.

### Canon et Product Forge

**Canon** est présenté comme une famille de workflows basés sur des repères, avec des modes comme *spec-first*, *code-first* et *spec-drift*. Cela permet de choisir un cadre stable pour contrôler l'écart entre l'intention écrite et le code produit.

**Product Forge** est l'extension orientée produit qui couvre le cycle complet : **research** → **product-spec** → **plan** → **tasks** → **implement** → **verify** → **test** → **release-readiness**. Autrement dit, on ne se limite pas à coder ; on structure aussi la découverte, la validation et la préparation de sortie .

### Le rôle du flux AIDE

Dans ce chapitre, AIDE sert à transformer une idée produit en chemin de livraison concret. La documentation de Spec Kit insiste sur des phases distinctes avec des contrôles de validation avant de passer à l'étape suivante, ce qui réduit les dérives de contexte et les décisions prématurées.

1. **Research** : on clarifie le besoin, les contraintes et les utilisateurs.
2. **Product-spec** : on formalise ce que le produit doit faire.
3. **Plan** : on précise comment l'équipe va le construire.
4. **Tasks** : on découpe le travail en éléments exécutables.
5. **Implement** : l'agent produit le code à partir des artefacts validés.

### Comment l'explorer concrètement

L'exploration commence par l'installation du bon extensionnement via le catalogue communautaire, puis par l'activation du workflow choisi dans l'environnement de l'agent IA. Spec

Kit permet de rechercher, ajouter et activer des extensions, et les commandes associées sont enregistrées automatiquement pour l'intégration d'agent utilisée.

- Utilisez **Canon Core** si vous voulez un cadre proche du workflow original adapté à Canon.
- Utilisez **Product Forge** si vous voulez un orchestrateur produit complet avec étapes produit, planification et contrôle qualité.
- Combinez-les si vous cherchez un pipeline plus strict, où la spec pilote vraiment l'implémentation.

## Exemple d'usage pédagogique

Imaginez une fonctionnalité de tableau de bord pour une équipe interne. Avec Product Forge, la phase **research** sert à identifier les attentes réelles, la phase **product-spec** décrit les usages, puis **plan** fixe l'architecture avant que **tasks** ne transforme tout cela en travaux atomiques .

Le point clé est simple : **AIDE ne remplace pas la spécification, il la prolonge**. Canon stabilise le cadre, Product Forge organise le cycle produit, et Spec Kit fournit les commandes et les artefacts pour faire avancer le projet sans perdre la cohérence entre intention et livraison.

## 51. Publier et partager un bundle réutilisable

### Publier un bundle réutilisable

Un **bundle** Spec Kit est un paquet versionné qui regroupe des **extensions**, **presets**, **steps** et **workflows** dans une configuration prête à réutiliser pour un rôle donné, comme développeur ou product manager. Il est décrit par un fichier `bundle.yml` écrit à la main, qui fige chaque composant sur une version précise

La publication commence par construire ce paquet de façon propre et vérifiable. Spec Kit fournit une commande de validation pour contrôler la structure et les références du bundle, puis une commande de build pour produire un artefact `.zip` versionné, ce qui facilite le partage et la reproductibilité entre équipes

### Préparer le manifeste

Le cœur du bundle est le fichier `bundle.yml`. Ce manifeste sert de contrat: il indique quels composants sont inclus, à quelles versions ils sont attachés, et parfois pour quelle **intégration** d'agent ils sont destinés. Quand aucune intégration n'est indiquée, le bundle est **agnostique**, c'est-à-dire qu'il s'adapte à l'intégration déjà utilisée par le projet

Un bundle bien préparé doit donc être lisible et explicite. En pratique, cela veut dire:

- choisir un rôle cible clairement défini;
- fixer les versions des composants;
- vérifier que chaque référence existe;
- éviter les dépendances implicites qui rendent le partage fragile

L'objectif n'est pas d'empiler des fichiers, mais de livrer un ensemble cohérent que quelqu'un d'autre peut installer sans deviner vos intentions.

### Valider avant de diffuser

La validation est l'étape qui évite de distribuer un bundle cassé. Spec Kit propose `specify bundle validate --path ./my-bundle` pour effectuer des contrôles structurels et de référence avant publication, ce qui réduit les erreurs au moment de l'installation chez un autre utilisateur

Un bon réflexe consiste à tester le bundle comme un produit fini. Par exemple, avant de le partager à une équipe, vérifiez qu'il contient bien les bons éléments, que les noms sont stables et que les versions pointent vers des composants réellement disponibles

## Construire et partager

Une fois validé, le bundle est construit avec `specify bundle build --path ./my-bundle`, qui génère l'archive versionnée à diffuser. Cette archive devient un artefact réutilisable: elle peut être transmise à une équipe, stockée dans un dépôt interne, ou intégrée à un processus de distribution plus large.

Pour le partage, la logique est simple:

1. valider le bundle;
2. générer l'archive;
3. diffuser la version produite;
4. demander aux utilisateurs de s'y tenir pour garder le même comportement

Cette approche est utile quand plusieurs personnes doivent suivre le même processus assisté par IA, car elle évite les variations entre postes, agents ou configurations.

## Donner envie de réutiliser

Un bundle réutilisable doit être pensé comme une *boîte à outils prête à brancher*, pas comme un brouillon personnel. Plus il est clair sur son objectif, plus il sera facile à adopter par d'autres équipes, surtout quand il combine des composants curés pour un rôle précis.

Pour faciliter l'adoption, documentez brièvement:

- le rôle visé;
- l'intégration cible ou le caractère agnostique;
- la version du bundle;
- la façon de valider et de reconstruire le paquet

Cette discipline transforme un ensemble local de réglages en un objet partageable, stable et exploitable dans le temps.

## 52. Automatiser de bout en bout en flux guidé

### Automatiser un flux de bout en bout

Un **flux guidé** est une séquence d'étapes où chaque action est définie à l'avance, avec un déclencheur, des règles et un résultat attendu. Dans Spec-Kit, ce mode sert à transformer une spécification claire en exécution automatisée, du besoin initial jusqu'à la livraison finale

### Définir le squelette du processus

Commencez par écrire une spécification courte mais précise : *entrée, sortie, critères de succès* et *conditions d'échec*. Les workflows d'IA sont justement conçus comme des pipelines de bout en bout, capables d'automatiser des tâches ou d'éclairer la décision avec du machine learning, du deep learning ou de la GenAI.

- **Déclencheur** : événement qui lance le flux.
- **Étape** : action intermédiaire, comme analyser, enrichir ou valider.
- **Sortie** : résultat final attendu.
- **Garde-fou** : règle qui bloque ou redirige si la qualité n'est pas suffisante.

### Orchestrer les étapes avec Spec-Kit

Le principe est de faire travailler l'IA *par étapes*, pas en une seule réponse. Cette approche réduit les dérives, car chaque phase reçoit seulement le contexte utile. Des guides de workflow agentique recommandent de séparer planification, implémentation et test, puis de transférer uniquement les sorties nécessaires entre sessions.

Spécification -> Plan -> Implémentation -> Vérification -> Livraison

Pensez à ce flux comme à une chaîne de montage : chaque poste fait une seule chose, puis transmet un objet propre au poste suivant.

Exemple simple : une demande de fonctionnalité arrive, Spec-Kit génère le plan, un agent produit le code, un autre vérifie la cohérence avec la spec, puis le flux publie uniquement si les critères sont remplis.

### Tester avant d'automatiser complètement

Un flux guidé efficace se teste d'abord en environnement contrôlé. Les recommandations de workflow IA insistent sur des simulations progressives, des tests unitaires de chaque étape et un test de bout en bout avec un cas standard avant tout déploiement complet.

- Tester chaque étape séparément.
- Vérifier que les transitions de données sont correctes.
- Contrôler les erreurs de contexte, de format et de logique.
- Ajouter une validation humaine sur les points critiques.

## **Mettre en place les contrôles opérationnels**

L'automatisation de bout en bout n'est fiable que si elle prévoit des conditions d'escalade. Les pratiques recommandées incluent des règles explicites, des seuils, des alertes et des mécanismes de reprise en cas d'échec, afin d'éviter qu'une mauvaise entrée se propage dans tout le pipeline.

Exemple concret :

- Si la spec est incomplète, le flux s'arrête.
- Si le code généré ne respecte pas le contrat d'API, le flux revient à l'étape de correction.
- Si le test final échoue, rien n'est publié.

Cette logique est particulièrement utile quand le processus comporte des tâches répétitives et multi-étapes, car l'IA peut gérer la partie routinière tandis que l'humain garde le contrôle sur les décisions sensibles .



## 53. Héberger catalogues privés et recommandations

### Héberger un catalogue privé

Un **catalogue privé** est une bibliothèque interne de spécifications, de composants ou de recommandations réservée à une équipe ou à une organisation. Dans un contexte Spec-Kit, il sert à centraliser les artefacts approuvés pour que les projets d'IA réutilisent des éléments validés au lieu de repartir de zéro.

Héberger ce catalogue dans un espace privé permet de contrôler qui voit quoi, de versionner les contenus et de limiter les dérives de conception. Les catalogues documentaires montrent déjà cette logique: un portail central agrège plusieurs sources, facilite la recherche et donne accès au bon contenu sans disperser l'information.

### Choisir l'architecture d'hébergement

Pour un usage organisationnel, l'approche la plus robuste consiste à séparer le **stockage**, l'**index de recherche** et l'**accès applicatif**. Le stockage conserve les fiches, modèles et recommandations; l'index permet de retrouver rapidement un élément par mot-clé; l'application expose l'interface aux équipes. Cette séparation reflète la logique des grands catalogues collectifs, qui fédèrent plusieurs bases tout en conservant leurs accès propres.

Les options courantes sont simples:

- **Portail web interne** pour consulter et filtrer les catalogues.
- **API privée** pour alimenter automatiquement les outils Spec-Kit.
- **Répertoire versionné** pour garder l'historique des recommandations et des bundles.

Un bon hébergement doit aussi gérer les métadonnées: auteur, version, domaine d'usage, niveau de maturité et date de validité. Les catalogues professionnels indiquent souvent des périodes de validité ou des règles d'accès afin d'éviter l'usage de contenus obsolètes.

### Organiser les contenus et les recommandations

Une recommandation utile doit être **actionnable**: elle explique quoi utiliser, dans quel cas, et avec quelles limites. Dans les catalogues privés orientés Spec-Kit, cela signifie lier chaque recommandation à une spécification, un exemple et un critère de validation.

Pour garder un catalogue exploitable:

- regroupez les contenus par domaine fonctionnel;
- ajoutez des mots-clés en français et en anglais;
- séparez les éléments stables des éléments expérimentaux;

- marquez clairement les dépendances entre modèles, prompts et règles.

Cette organisation ressemble aux catalogues collectifs qui orientent la recherche à partir d'une notice, d'un nom de bibliothèque ou d'un filtre précis. Plus la structure est claire, plus les équipes trouvent vite la bonne ressource.

## Sécuriser l'accès et la gouvernance

Un catalogue privé doit être protégé par une authentification forte, un contrôle d'accès par rôle et une journalisation des consultations. Le principe est simple: seuls les utilisateurs autorisés voient les recommandations, et chaque modification doit être traçable.

Les bonnes pratiques de gouvernance comprennent:

- revue avant publication;
- validation par un référent technique;
- archivage des versions remplacées;
- suppression ou mise en quarantaine des contenus non conformes.

L'objectif n'est pas seulement de restreindre l'accès, mais d'assurer la confiance dans ce qui est publié. Un catalogue utile est un catalogue où l'on sait *qui a publié quoi, quand, et pourquoi*.

## Exemple de mise en œuvre Spec-Kit

Un schéma efficace pour une équipe produit consiste à créer un espace privé avec trois collections: `specs/`, `patterns/`, `recommendations/`. Chaque entrée contient un titre, un résumé, une version, un statut et un exemple d'usage.

```
title: Authentification API
version: 1.3
status: approved
use_case: services internes
recommendation: "Utiliser OAuth 2.1 pour les appels serveur-à-serveur"
example: "token exchange"
```

Les équipes consultent ensuite le catalogue via une interface interne ou une API, puis réutilisent la recommandation dans leurs spécifications. Ce mode de fonctionnement réduit les écarts entre projets, accélère les décisions et rend les choix d'architecture plus cohérents, comme dans les grands catalogues qui fédèrent plusieurs sources pour simplifier la recherche.

## 54. Travailler hors ligne derrière pare-feu

### Travailler sans Internet

Un environnement *hors ligne* signifie que la machine de développement n'a pas d'accès direct à Internet. Derrière un **pare-feu**, les flux entrants et sortants sont filtrés par des règles ; un pare-feu est justement un système de sécurité placé à la périphérie du réseau pour inspecter le trafic et bloquer les paquets suspects .

Dans ce contexte, l'enjeu est simple : **garder Spec-Kit utilisable sans dépendre du réseau**. Cela demande de préparer à l'avance les outils, les modèles, et les dépôts nécessaires, puis de travailler avec des artefacts locaux, comme on le ferait avec une boîte à outils déjà ouverte.

### Préparer l'espace de travail

Avant de lancer un projet, installez les composants nécessaires dans un environnement contrôlé. Spec-Kit propose un processus guidé avec des commandes séparées pour définir les principes du projet, spécifier le besoin, planifier, découper en tâches, implémenter, puis converger vers la spécification.

En pratique, cela signifie qu'en mode isolé vous devez :

- conserver le dépôt Spec-Kit et ses dépendances locales ;
- garder une copie des modèles d'IA ou du moteur compatible si votre organisation en utilise un ;
- stocker les spécifications de projet dans un répertoire partagé interne ;
- vérifier que chaque commande peut s'exécuter sans téléchargement supplémentaire .

Cette préparation réduit les blocages quand le pare-feu empêche l'accès aux services externes.

### Configurer le pare-feu pour un usage maîtrisé

Un pare-feu ne doit pas seulement “bloquer” ; il doit **autoriser précisément** ce qui est nécessaire. Les recommandations de l'ANSSI distinguent un pare-feu externe, connecté à Internet, et un pare-feu interne, connecté au système à protéger ; la zone entre les deux est la **DMZ**.

Pour travailler hors ligne, la logique est comparable :

- autoriser seulement les échanges vers les dépôts internes ;
- interdire les accès Internet non nécessaires ;
- isoler les postes de développement du reste du SI ;

- journaliser les tentatives de connexion pour repérer les écarts.

Un bon filtre réseau ne sert pas à “tout fermer”, mais à rendre chaque autorisation explicite.

## Utiliser Spec-Kit en réseau fermé

Spec-Kit repose sur un flux structuré : *constituer, spécifier, planifier, découper, implémenter*, puis *converger*. En environnement protégé, gardez cette logique mais remplacez les dépendances externes par des ressources locales.

Exemple concret :

```
/speckit-constitution  
/speckit-specify  
/speckit-plan  
/speckit-tasks  
/speckit-implement  
/speckit-converge
```

Si votre agent ne peut pas joindre un service distant, préparez les réponses attendues dans des fichiers internes. Cette méthode évite les interruptions et permet de travailler même lors d’une coupure réseau temporaire .

## Bonnes pratiques d’exploitation

En réseau fermé, la sécurité repose aussi sur l’hygiène opérationnelle. Les guides de configuration recommandent de supprimer les comptes par défaut, d’utiliser des mots de passe uniques, de limiter les privilèges et de tester les règles après chaque changement.

Pour un usage Spec-Kit, retenez ces règles :

- versionner localement vos spécifications et plans ;
- sauvegarder les configurations du poste et du pare-feu ;
- tester l’accès aux ressources internes avant chaque session ;
- documenter les exceptions réseau demandées par l’équipe ;
- contrôler les journaux pour vérifier que rien ne sort du périmètre autorisé .

Cette discipline rend le développement IA plus prévisible et plus simple à exploiter dans une organisation contrainte par un pare-feu.

## 55. Gouverner conformité et bonnes pratiques

### Définir la gouvernance dès la spécification

La **gouvernance** désigne l'ensemble des règles, rôles et preuves qui encadrent un système d'IA. Dans une démarche Spec-Kit, elle doit apparaître dès la spécification, car la CNIL rappelle qu'un système d'IA qui collecte ou utilise des données personnelles doit respecter le RGPD et les droits des personnes.

Un bon réflexe consiste à transformer chaque exigence en contrainte vérifiable . Par exemple, au lieu d'écrire "le système doit être sécurisé", précisez "l'accès doit être limité aux personnes autorisées, avec traçabilité des actions". Microsoft recommande justement une journalisation des interactions IA et des évaluations de confidentialité pour documenter les projets.

### Construire un dossier de conformité exploitable

La conformité ne repose pas sur une promesse, mais sur des éléments concrets : **inventaire des systèmes, propriétaire, objectif, niveau de risque, preuves et historique des décisions**. Snowflake indique qu'un programme de conformité IA commence souvent par un inventaire, une fiche modèle, un résumé des données d'entraînement et un registre des approbations.

Dans Spec-Kit, cela se traduit par un fichier de spécification qui contient :

- le cas d'usage exact ;
- les données traitées ;
- les personnes concernées ;
- les contrôles attendus ;
- les tests de validation ;
- les limites d'usage.

La CNIL propose aussi une grille d'analyse et un guide d'auto-évaluation pour aider les organismes à apprécier la maturité de leurs systèmes au regard du RGPD.

### Appliquer les bonnes pratiques de contrôle

Les bonnes pratiques de conformité doivent être pensées comme des **contrôles** : des mécanismes qui réduisent un risque précis. Le code de bonnes pratiques de l'IA à usage général de l'Union européenne vise la sécurité, la transparence et le droit d'auteur pour les modèles GPAI.

Concrètement, les équipes peuvent exiger :

- une **identité authentifiée** pour chaque agent IA ;
- un **accès limité** au strict nécessaire ;
- un **journal d’audit** infalsifiable ;
- une **documentation** à jour des versions et des tests.

Kiteworks souligne que la gouvernance de l’IA converge souvent vers ces mêmes contrôles : authentification, accès régulé, chiffrement et journalisation.

## Intégrer la conformité dans le cycle de validation

La conformité doit être vérifiée *avant* le déploiement, pas après. La CNIL recommande d’informer clairement les personnes concernées et de documenter le traitement, tandis que Microsoft conseille de conserver les détails du projet IA tout au long du développement.

Dans un flux Spec-Kit, chaque exigence de conformité peut devenir une clause testable. Exemple : “toute réponse produite par l’IA doit pouvoir être reliée à une version de modèle et à un journal d’exécution”. Cela rend la preuve plus simple à produire en cas d’audit.

## Ancrer les bonnes pratiques dans l’usage quotidien

Les équipes doivent aussi adopter des règles simples d’usage. Le gouvernement du Québec recommande de ne jamais divulguer d’informations confidentielles, de vérifier les conditions d’utilisation, de chiffrer les communications et de rester vigilant face aux erreurs possibles des outils d’IA générative.

Pour industrialiser ces pratiques, une organisation peut formaliser :

- un guide utilisateur court ;
- une revue de conformité avant mise en production ;
- un registre des incidents et alertes ;
- une mise à jour régulière des spécifications.

La CNIL rappelle enfin qu’une démarche sérieuse de conformité inclut la prise en compte du risque, la documentation continue et la responsabilité attribuée à un acteur identifié.

## 56. Adopter Spec-Kit sur projet existant

### Préparer l'adoption sur un dépôt déjà vivant

Adopter Spec-Kit sur un projet existant signifie *brancher* le flux piloté par spécifications sur une base de code déjà en production ou en évolution, sans repartir de zéro. GitHub recommande de d'abord **committer** ou **stasher** le travail en cours, puis de créer une branche dédiée à l'adoption avant toute initialisation.

Ensuite, initialisez le dépôt **à l'endroit même du projet** avec la commande `specify init --here --force --integration <key>` depuis la racine du repository, en choisissant la clé d'intégration correspondant à l'agent de codage utilisé. Cette étape prépare les fichiers et commandes Spec-Kit dans le contexte réel du code existant, ce qui évite de créer un second projet parallèle .

### Capturer les règles qui protègent l'existant

Le cœur de l'adoption est de formaliser les **guardrails**, c'est-à-dire les règles déjà vraies pour le dépôt ou explicitement acceptées par l'équipe. GitHub indique de lancer `/speckit.constitution` pour enregistrer ces principes comme base de travail. Une *constitution* est un ensemble de contraintes stables, par exemple les conventions de test, les limites d'architecture ou les exigences de compatibilité.

Cette étape est essentielle sur un projet existant, car elle permet à l'agent IA de respecter le code déjà en place au lieu de proposer une refonte implicite. Une bonne règle de départ est simple : *si la pratique existe déjà dans le dépôt et reste souhaitée, elle doit apparaître dans la constitution* .

### Choisir une première modification bornée

Spec-Kit fonctionne mieux lorsqu'on commence par un **changement borné**, c'est-à-dire une fonctionnalité, correction ou modernisation qui peut être revue seule. GitHub précise de ne pas faire de “documenter tout le système existant” votre première demande, sauf si l'inventaire complet est lui-même le livrable attendu.

Exemple utile : ajouter un nouvel écran, corriger un bug précis, ou moderniser un module isolé. Cette approche réduit le risque de dérive et rend la validation plus simple pour l'équipe. Elle aide aussi à mesurer rapidement si Spec-Kit s'intègre bien au rythme du projet existant.

## Planifier à partir du dépôt réel

Une fois le premier changement choisi, on suit le flux normal : `/speckit.clarify`, puis `/speckit.plan`, `/speckit.tasks`, `/speckit.analyze`, `/speckit.implement`, et enfin `/speckit.converge`. La clarification sert à lever les ambiguïtés fonctionnelles et de compatibilité ; la planification doit ensuite réutiliser l'architecture, les dépendances et les conventions de tests déjà présentes dans le dépôt.

Sur un projet existant, le bon plan n'impose pas un monde nouveau : il *s'aligne* sur le monde réel du repository.

Les tâches doivent rester cohérentes avec l'existant, puis l'analyse vérifie cette cohérence avant l'implémentation. Si la convergence révèle des écarts, on recommence le cycle jusqu'à ce que la modification soit complète.

## Décider comment les spécifications vivent après coup

Après la première adoption, l'équipe doit fixer la manière dont les artefacts Spec-Kit seront conservés dans le temps. GitHub propose trois options : garder chaque dossier de feature comme archive historique immuable, maintenir `spec.md` comme contrat vivant avec régénération des artefacts dérivés, ou faire remonter les découvertes du code vers les plans et spécifications pour réconcilier l'ensemble.

Le bon choix dépend de votre organisation, mais la règle pratique est claire : sur un projet existant, il faut éviter que les specs deviennent des documents oubliés . Elles doivent rester reliées au code, aux tâches et aux décisions réelles, sinon l'adoption perd son intérêt opérationnel.

## Exemple d'enchaînement concret

1. Créer la branche d'adoption.
2. Lancer `specify init --here --force --integration <key>`.
3. Rédiger `/speckit.constitution`.
4. Choisir une petite évolution bien délimitée.
5. Exécuter `/speckit.clarify`, puis `/speckit.plan` et `/speckit.tasks`.
6. Vérifier avec `/speckit.analyze`.
7. Implémenter et converger avec `/speckit.implement` puis `/speckit.converge`.

Ce déroulé transforme l'adoption en *expérience contrôlée* plutôt qu'en migration risquée, ce qui est précisément ce qu'on recherche sur une base de code déjà utile et déjà fragile.